DaTra: Data Transformation Semantics as a Symmetric Monoidal Category on Atlas Presheaves

@qualiascript ChatGPT

Abstract

This work formalizes the semantics of the Datra programming-language project in Lean. Data domains are organized into atlases: structured, eventually stable families of pages encoding how data is partitioned and related. Data transformations are modeled as presheaves on the resulting atlas categories, yielding a topos-theoretic semantics. Stable transformations are then assembled over atlas federations, whose horizontal sum extends by Day convolution to a symmetric monoidal category.

1 Dominions

Definition 1.1 (The Category of Dominions)**.** The **Category of Dominions**, denoted Dom, has as its objects sets whose cardinality is at most ω_0, equivalently sets admitting an injection into ω_0, and as its morphisms set-theoretic functions.

Definition 1.2 (The Domanial Embedding Functor)**.** The **Domanial Embedding Functor**, denoted $\mathsf{Emb} : \mathsf{Dom} \to \mathsf{Set}$, sends each dominion to its underlying set.

2 Domanial Insertions

Definition 2.1 (The Category of Domanial Insertions)**.** The **Category of Domanial Insertions**, denoted DomIns, has dominions as objects and monomorphisms in Dom as morphisms.

Definition 2.2 (The Category of Codomanial Insertions)**.** The **Category of Codomanial Insertions**, denoted $\mathsf{CoDomIns}$, is the opposite category $\mathsf{DomIns}^{\mathrm{op}}$.

3 Chains

Definition 3.1 (Chain)**.** A **Chain** is a totally ordered thin category, indexed skeletally by an ordinal strictly smaller than $\omega_0^{\omega_0}$.

Definition 3.2 (The Spine)**.** The **spine** is the chain with ω_0 objects.

4 Consolidations

Definition 4.1 (The Category of Consolidations)**.** The **Category of Consolidations**, denoted Con, has chains as objects and point-surjective functors between their thin categories as morphisms.

Definition 4.2 (The Category of Coconsolidations)**.** The **Category of Coconsolidations**, denoted CoCon, is the opposite category $\mathsf{Con}^{\mathrm{op}}$.

5 Transportation and Folios

Definition 5.1 (The Transportation Functor)**.** The **Transportation Functor**, denoted $\mathsf{Tra} : \mathsf{Con} \to \mathsf{Set}$, sends a chain to its object set and a consolidation to its underlying set-theoretic surjection.

Definition 5.2 (Folio)**.** A **folio** is a functor $F : S \to \mathsf{CoCon}$, where S is the spine, such that $F(0)$ is the singleton chain.

6 Paginations

Definition 6.1 (The Category of Paginations)**.** The **Category of Paginations**, denoted Pag, has as objects pairs (H, E) where $H = \mathsf{Tra} \circ F^{\mathrm{op}}$ for a folio F, and E is the category of elements of H. For $W : \mathsf{Pag}$, write W_H for the first inclusion and W_E for the second inclusion. A morphism $T : X \to Y$ in Pag is a functor $T : X_E \to Y_E$. Thus, for every morphism $f : x \to y$ of X_E, the following square commutes:

$$\begin{array}{ccc} x = (m,i) & \xrightarrow{\quad f \quad} & (n, X_H(f)(i)) \\ {\scriptstyle T}\downarrow & & \downarrow{\scriptstyle T} \\ (m', i') & \xrightarrow[\quad T(f) \quad]{} & (n', Y_H(T(f))(i')). \end{array}$$

7 Atlases

Definition 7.1 (The Category of Atlases)**.** The **Category of Atlases**, denoted Atl, has as objects pairs (P, G), where $P : \mathsf{Pag}$ and $G : P_E \to \mathsf{DomIns}$ is a functor, equivalently a presheaf $G : P_E^{\mathrm{op}} \to \mathsf{CoDomIns}$. For $P_H : C \to \mathsf{Set}$, $m \in |C|$, and distinct elements $k, k' \in |P_H(m)|$, the pullback in G of the arrows induced by $(m \to 0)(k)$ and $(m \to 0)(k')$ is empty.

For every $n \in \mathbb{N}$, the n**th atlas page** of X is the set $X_H(n)$ on its full spine. An element $k \in X_H(n)$ is a **page cell**, written (n, k), and the dominion carried by that cell is $X_G(n, k)$. Thus a page is the indexed collection of cells at one spine position.

There is an integer w such that, for every $w' > w$, $P_H(w' \to w) = \mathrm{Id}$, and, for every $k \in |P_H(w)|$, the corresponding arrow $G((w' \to w)(k))$ is the identity. The least such integer is the **cardinality of the atlas**, denoted $|A|$.

For $X : \mathsf{Atl}$, write X_P and X_G for the two components and put $X_H = (X_P)_H$ and $X_E = (X_P)_E$. A morphism $T : X \to Y$ is a pair (T_P, T_A), where $T_P : X_P \to Y_P$ is a pagination morphism and $T_A : X_G \Rightarrow Y_G \circ T_E$ is a natural transformation. Thus, for every $f : x \to y$ in X_E, the following square commutes:

$$\begin{array}{ccc} X_G(x) & \xrightarrow{\quad X_G(f) \quad} & X_G(y) \\ {\scriptstyle (T_A)_x}\downarrow & & \downarrow{\scriptstyle (T_A)_y} \\ Y_G(T_E(x)) & \xrightarrow[\quad Y_G(T_E(f)) \quad]{} & Y_G(T_E(y)). \end{array}$$

Finally, if $x = |X|$, $x' > x$, and $r \in |X_H(|X| - 1)|$, then $T_P(x', r) = T_P(x, r)$ and $(T_A)_{(x',r)} = (T_A)_{(x,r)}$.

Definition 7.2 (Extent of an Atlas)**.** The **extent** of A is $\mathsf{Ex}(A) = A_G(0, 0)$. Since objects of DomIns are dominions, $\mathsf{Ex}(A) : \mathsf{Dom}$ for every $A : \mathsf{Atl}$.

Definition 7.3 (Territory of an Atlas)**.** Let $M = A_H(|A| - 1)$. The **territory** is the indexed family $\mathsf{Ter}(A) : M \to \mathsf{Dom}$ given by $\mathsf{Ter}(A)(k) = A_G(|A| - 1, k)$.

Definition 7.4 (nth Region of an Atlas)**.** The **nth region** of A is $\mathsf{Ter}(A)(n)$, with indexing beginning at 0.

8 Transposals and Traversals

Definition 8.1 (The Category of Atlas Transposals)**.** The **Category of Atlas Transposals**, denoted $\mathsf{AtlTrap}$, is the wide subcategory of Atl whose morphisms $F : X \to Y$ have point-injective object maps F_E.

Definition 8.2 (The Category of Atlas Traversals)**.** The **Category of Atlas Traversals**, denoted $\mathsf{AtlTrav}$, is the wide subcategory of $\mathsf{AtlTrap}$ whose morphisms satisfy the following conditions. For $F : X \to Y$, let $x = (m, i)$ and $y = (m', j)$, put $p = \min(m, m')$, write $F_E(x) = (n, i')$ and $F_E(y) = (n', j')$, and put $p' = \min(n, n')$. If

$$X_H(m \to p)(i) < X_H(m' \to p)(j),$$

then

$$Y_H(n \to p')(i') < Y_H(n' \to p')(j').$$

Furthermore, if $q \in |X|$ and $t \in X_G(q)$ is in the image of a final region—that is, there exist $k \in |\mathsf{Ter}(X)|$ and $l \in \mathsf{Ter}(X)(k)$ whose image under $X_G((|X| - 1) \to q)(k)$ is t—then this property is preserved by F.

Definition 8.3 (The Category of Stable Atlas Traversals)**.** The **Category of Stable Atlas Traversals**, denoted $\mathsf{StaAtlTrav}$, is the wide subcategory of $\mathsf{AtlTrav}$ whose morphisms $F : X \to Y$ satisfy $F_E(0, 0) = (0, 0)$.

9 Maps and Cartography

Definition 9.1 (The Category of Atlas Maps)**.** The **Category of Atlas Maps**, denoted AtlMap, is the full subcategory of atlases for which every element of the extent is in the image of a final region.

Definition 9.2 (The Atlas Map Inclusion Functor)**.** The **Atlas Map Inclusion Functor** $\mathsf{AtlMapInc} : \mathsf{AtlMap} \to \mathsf{Atl}$ is the canonical inclusion.

Definition 9.3 (The Category of Atlas Traversal Maps)**.** The **Category of Atlas Traversal Maps**, denoted $\mathsf{AtlTravMap}$, is the full subcategory of $\mathsf{AtlTrav}$ whose objects are Atlas Maps.

Definition 9.4 (The Atlas Traversal Map Inclusion Functor)**.** The canonical inclusion is denoted $\mathsf{AtlTravMapInc} : \mathsf{AtlTravMap} \to \mathsf{AtlTrav}$.

Lemma 9.5 (Cartography Lemma)**.** *The inclusion* $\mathsf{AtlTravMapInc}$ *has a right adjoint, denoted the* ***Charting Functor*** $\mathsf{Chr} : \mathsf{AtlTrav} \to \mathsf{AtlTravMap}$*.*

Proof sketch. *On an object* $X : \mathsf{AtlTrav}$*, the functor* Chr *sends* X *to the universal arrow from* $\mathsf{AtlTravMapInc}$ *to* X*. Write* $X' = \mathsf{AtlTravMapInc}(\mathsf{Chr}(X))$*, together with* $H : X' \to X$*, with* X' *terminal among objects having this property. The solution is the DaTra map satisfying* $\mathsf{Ter}(X') \cong \mathsf{Ter}(X)$*. It is unique because* $\mathsf{AtlTrav}$ *preserves orders and terminal because* H_E *is faithful.*

10 Coalitions

Definition 10.1 (The Coalizing Functor)**.** The **Coalizing Functor**, denoted

$$\mathsf{Coa} : \mathsf{StaAtlTrav} \longrightarrow \mathsf{DomIns},$$

is the extent of the charted atlas: $\mathsf{Coa} = \mathsf{Ex} \circ \mathsf{Chr}$.

Definition 10.2 (The Coalition of an Atlas)**.** The **coalition** of an atlas X is the extent of its chart, regarding X as an object of the wide category of stable traversals.

Definition 10.3 (The Domanial Inclusion Functor)**.** The **Domanial Inclusion Functor** $\mathsf{DomInc} : \mathsf{DomIns} \to \mathsf{StaAtlTrav}$ sends a dominion to the atlas of cardinality 1 whose coalition is that dominion. It is left adjoint to Coa: for $X : \mathsf{DomIns}$ and $Y : \mathsf{StaAtlTrav}$, naturally in X and Y,

$$\mathrm{Hom}_{\mathsf{StaAtlTrav}}(\mathsf{DomInc}(X), Y) \cong \mathrm{Hom}_{\mathsf{DomIns}}(X, \mathsf{Coa}(Y)),$$

and $\mathsf{Coa}(\mathsf{DomInc}(X)) \cong X$.

11 Data Transformations

Definition 11.1 (The Category of Data Transformations)**.** The **Category of Data Transformations**, also called the category of **Data Transformation Sets** or **DaTra Sets**, is the presheaf category

$$\mathsf{DaTra} = [\mathsf{Atl}^{\mathrm{op}}, \mathsf{Set}].$$

As a presheaf category, it is a topos.

Definition 11.2 (Navigation)**.** A **navigation** of $D : \mathsf{DaTra}$ is a monomorphism $\mathsf{Nav} : \mathsf{Yo}(A) \to D$ for some atlas A.

Definition 11.3 (Expedition)**.** An **expedition** is a navigation represented by an Atlas Map.

Definition 11.4 (The Category of Data Transposals)**.** The **Category of Data Transposals**, denoted DaTrap, is the presheaf category $[\mathsf{AtlTrap}^{\mathrm{op}}, \mathsf{Set}]$.

Definition 11.5 (The Category of Data Traversals)**.** The **Category of Data Traversals**, denoted DaTrav, is the presheaf category $[\mathsf{AtlTrav}^{\mathrm{op}}, \mathsf{Set}]$.

Definition 11.6 (Data Transformation Maps)**.** The **Category of Data Transformation Maps**, denoted $\mathsf{DaTraMap}$, is the full subcategory of DaTra Sets all of whose navigations are expeditions.

12 Atlas Federations

Definition 12.1 (Atlas Merge)**.** The **Atlas Merge** is the object operation

$$\mathsf{AtlMerge} : \mathrm{Ob}(\mathsf{Atl}) \times \mathrm{Ob}(\mathsf{Atl}) \longrightarrow \mathrm{Ob}(\mathsf{Atl}).$$

For atlases X and Y, the extent of $\mathsf{AtlMerge}(X, Y)$ is the disjoint union of their extents. Both atlases are evaluated on their full spines: page $n+1$ of the merge is the tagged, componentwise combination of page n of X and page n of Y.

Definition 12.2 (Atlas Federation)**.** An **Atlas Federation** is a countable tagged family of atlas objects. The category of Atlas Federations is denoted

$$\mathsf{AtlFed}.$$

A morphism consists of a map of tags and, at each source tag, a stable atlas traversal to its selected target tag.

Definition 12.3 (The Empty Atlas)**.** The **Empty Atlas** is $\mathsf{AtlI} = \mathsf{DomInc}(\varnothing)$, also viewed as an Atlas Federation.

Definition 12.4 (The Atlas Horizontal Sum Bifunctor)**.** The **Atlas Horizontal Sum Bifunctor**, denoted $\mathsf{AtlHorSum} : \mathsf{AtlFed} \times \mathsf{AtlFed} \to \mathsf{AtlFed}$, has object action

$$\mathsf{AtlHorSum}(X, X') = \mathsf{AtlMerge}(X, X').$$

Here AtlFed is the category of Atlas Federations; its empty federation represents AtlI. Given stable traversals $F : X \to Y$ and $F' : X' \to Y'$, horizontal sum preserves the left and right tags and applies F and F' componentwise. Stability fixes the common origin, so this arrow action is again a stable atlas traversal.

Lemma 12.5 (Horizontal Lemma)**.** *The Atlas Horizontal Sum Bifunctor* $\mathsf{AtlHorSum}$ *exists.*

Proof. *Identity arrows act identically on every tag, composition holds componentwise, and every component arrow is required to lie in* $\mathsf{StaAtlTrav}$*. The federation structure supplies the associator, unitors, and braider by reassociation, deletion of the empty tag family, and tag swapping, respectively.*

Definition 12.6 (The Atlas Braider)**.** The **Atlas Braider** $\mathsf{AtlBrd}_{F,F'} : \mathsf{AtlHorSum}(F, F') \to \mathsf{AtlHorSum}(F', F)$ swaps the left and right component tags. Hence $\mathsf{AtlBrd}_{F,F'} \circ \mathsf{AtlBrd}_{F',F} = \mathrm{Id}$.

Definition 12.7 (The Atlas Associator)**.** The **Atlas Associator**, denoted

$$\mathsf{AtlAsoc}_{F,F',F''} : \mathsf{AtlHorSum}(\mathsf{AtlHorSum}(F, F'), F'') \longrightarrow \mathsf{AtlHorSum}(F, \mathsf{AtlHorSum}(F', F'')),$$

flattens the nested federation and retags its three sides using the canonical reassociation $((x + y) + z) \leftrightarrow (x + (y + z))$. Its data component is the matching reassociation of the three extent summands.

Definition 12.8 (The Atlas Unitors)**.** The **Atlas Left Unitor** $\mathsf{Lu} : \mathsf{AtlHorSum}(\mathsf{AtlI}, F) \to F$ and the **Atlas Right Unitor** $\mathsf{Ru} : \mathsf{AtlHorSum}(F, \mathsf{AtlI}) \to F$ remove the empty tag family.

13 Stable Data Transversals

Definition 13.1 (Stable Data Transversals)**.** The **Category of Stable Data Transversals**, denoted $\mathsf{StaDaTrav}$, is the presheaf category on Atlas Federations whose component arrows lie in $\mathsf{StaAtlTrav}$.

Definition 13.2 (The Empty Map)**.** The **Empty Map**, denoted I, is the Day unit represented by the empty Atlas Federation:

$$I = \mathsf{Yo}(\mathsf{AtlI}).$$

Definition 13.3 (The Horizontal Sum Bifunctor)**.** The **Horizontal Sum Bifunctor**, denoted $\mathsf{HorSum} : \mathsf{StaDaTrav} \times \mathsf{StaDaTrav} \to \mathsf{StaDaTrav}$, or in infix notation by $+_< : \mathsf{StaDaTrav} \times \mathsf{StaDaTrav} \to \mathsf{StaDaTrav}$, is the Day convolution extension of stable atlas horizontal sum. Because the indexing morphisms are arrows of $\mathsf{StaAtlTrav}$, its result and its arrow action land in $\mathsf{StaDaTrav}$ by construction.

Definition 13.4 (The Stable Data Transversal Braider)**.** The **Stable Data Transversal Braider** $\mathsf{Brd}_{F,F'} : F +_< F' \to F' +_< F$ is the Day convolution extension of AtlBrd.

Definition 13.5 (The Stable Data Transversal Associator)**.** The **Stable Data Transversal Associator**

$$\mathsf{Asoc}_{F,F',F''} : (F +_< F') +_< F'' \longrightarrow F +_< (F' +_< F'')$$

is the Day convolution extension of $\mathsf{AtlAsoc}$.

Definition 13.6 (The Stable Data Transversal Unitors)**.** The **Stable Data Transversal Left Unitor** $\mathsf{Lu} : I +_< F \to F$ and the **Stable Data Transversal Right Unitor** $\mathsf{Ru} : F +_< I \to F$ are the Day convolution extensions of AtlLu and AtlRu.

Definition 13.7 (Stable Data Transversals Monoidal Category)**.** The **Stable Data Transversals Monoidal Category**, denoted $\mathsf{StaDaTravMon}$, is

$$\mathsf{StaDaTravMon} = (\mathsf{StaDaTrav}, +_<, I).$$

Its tensor is Day convolution on the Atlas Federation. The braider, associator, and unitors are induced by retagging that form, so all coherence maps remain within stable data transversals.

A Lean Formalization

The following is an ASCII rendering of the complete Lean source corresponding to the mathematical development above. It replaces Lean's conventional Unicode surface notation solely for portable typesetting; the checked source file remains `datra.lean`.

```
import Mathlib

namespace Datra

open CategoryTheory CategoryTheory.Functor CategoryTheory.Limits Opposite
open scoped Ordinal

universe u

noncomputable section

/-- A dominion is a set equipped with a certificate of countability. -/
structure Dominion where
  Carrier : Type
  rank : Function.Embedding Carrier Nat

abbrev Dom := Dominion

instance : CoeSort Dominion Type where coe X := X.Carrier

instance : Category Dominion where
  Hom X Y := X -> Y
  id _ := id
  comp f g := g  o  f
  id_comp _ := rfl
  comp_id _ := rfl
  assoc _ _ _ := rfl

def Emb : Dom ==> Type where
  obj X := X
  map f := f

structure DomIns where
  toDom : Dom

instance : CoeSort DomIns Type where coe X := X.toDom.Carrier

instance : Category DomIns where
  Hom X Y := Function.Embedding X Y
  id X := Function.Embedding.refl X
  comp f g := f.trans g
  id_comp f := by ext; rfl
  comp_id f := by ext; rfl
  assoc f g h := by ext; rfl

instance {X Y : DomIns} : CoeFun (X --> Y) (fun _ => X -> Y) where
  coe f := f.toFun

@[ext]
theorem DomIns.hom_ext {X Y : DomIns} (f g : X --> Y)
    (h : forall  x, f x = g x) : f = g := by
  exact Function.Embedding.ext h

instance {X Y : DomIns} (f : X --> Y) : Mono f where
  right_cancellation g h w := by
    apply DomIns.hom_ext
    intro x
    apply f.injective
    exact congrFun (congrArg Function.Embedding.toFun w) x

def DomIns.forget : DomIns ==> Dom where
  obj X := X.toDom
  map f := f.toFun

abbrev CoDomIns := Opposite DomIns

```

```
/-- A chain carries its skeletal well-ordered object type and a proof that
its order type lies below `omega_0 ^ omega_0`.  Retaining the carrier explicitly makes
ordinal sums of chains definitionally usable. -/
structure Chain where
  Obj : Type
  linearOrder : LinearOrder Obj
  wellFoundedLT : WellFoundedLT Obj
  orderType_lt : Ordinal.type (fun x y : Obj => x < y) <
    Ordinal.omega0 ^ Ordinal.omega0

attribute [instance] Chain.linearOrder Chain.wellFoundedLT

def Chain.sum (X Y : Chain) : Chain where
  Obj := Lex (Sum X.Obj Y.Obj)
  linearOrder := inferInstance
  wellFoundedLT := <|by
    change WellFounded (Sum.Lex (fun x y : X.Obj => x < y)
      (fun x y : Y.Obj => x < y))
    exact Sum.lex_wf wellFounded_lt wellFounded_lt|>
  orderType_lt := by
    change Ordinal.type (Sum.Lex (fun x y : X.Obj => x < y)
      (fun x y : Y.Obj => x < y)) < Ordinal.omega0 ^ Ordinal.omega0
    rw [Ordinal.type_sum_lex]
    exact Ordinal.principal_add_omega0_opow Ordinal.omega0
      X.orderType_lt Y.orderType_lt

/-- The common page-indexing chain. -/
def Spine : Chain where
  Obj := Nat
  linearOrder := inferInstance
  wellFoundedLT := inferInstance
  orderType_lt := by
    have hpow : Ordinal.omega0 ^ (1 : Ordinal) <
        Ordinal.omega0 ^ Ordinal.omega0 :=
      (Ordinal.opow_lt_opow_iff_right Ordinal.one_lt_omega0).2
        Ordinal.one_lt_omega0
    change typeLT Nat < Ordinal.omega0 ^ Ordinal.omega0
    rw [Ordinal.type_nat_lt]
    simpa only [Ordinal.opow_one] using hpow

abbrev Con := Chain

/-- A functor between ordinal chains is equivalently a monotone object map. -/
structure ConHom (X Y : Con) where
  toFun : X.Obj -> Y.Obj
  monotone : Monotone toFun
  point_surjective : Function.Surjective toFun

def ConHom.sum {X X' Y Y' : Con} (f : ConHom X Y) (g : ConHom X' Y') :
    ConHom (Chain.sum X X') (Chain.sum Y Y') where
  toFun z := toLex (Sum.map f.toFun g.toFun (ofLex z))
  monotone := by
    intro a b h
    change Sum.Lex (fun x y : X.Obj => x <= y) (fun x y : X'.Obj => x <= y)
      (ofLex a) (ofLex b) at h
    change Sum.Lex (fun x y : Y.Obj => x <= y) (fun x y : Y'.Obj => x <= y)
      (Sum.map f.toFun g.toFun (ofLex a))
      (Sum.map f.toFun g.toFun (ofLex b))
    generalize ha : ofLex a = sa at h |-
    generalize hb : ofLex b = sb at h |-
    rcases sa with x | x <;> rcases sb with y | y
    .
      simp only [Sum.map]
      exact Sum.Lex.inl (f.monotone (Sum.lex_inl_inl.mp h))
    . simp only [Sum.map]
      exact Sum.Lex.sep _ _
    . exact (Sum.lex_inr_inl h).elim
    .
      simp only [Sum.map]
      exact Sum.Lex.inr (g.monotone (Sum.lex_inr_inr.mp h))
  point_surjective := by
    intro z
    rcases hz : ofLex z with y | y
```

```
    . obtain <|x, rfl|> := f.point_surjective y
      refine <|toLex (Sum.inl x), ?_|>
      apply toLex.injective
      simpa using hz.symm
    . obtain <|x, rfl|> := g.point_surjective y
      refine <|toLex (Sum.inr x), ?_|>
      apply toLex.injective
      simpa using hz.symm

instance {X Y : Con} : CoeFun (ConHom X Y) (fun _ => X.Obj -> Y.Obj) where
  coe f := f.toFun

@[ext]
theorem ConHom.ext {X Y : Con} (f g : ConHom X Y)
    (h : forall  x, f x = g x) : f = g := by
  cases f
  cases g
  simp_all only [mk.injEq]
  exact funext h

instance : Category.{0} Con where
  Hom := ConHom
  id X := <|id, monotone_id, Function.surjective_id|>
  comp f g := <|g  o  f, g.monotone.comp f.monotone,
    g.point_surjective.comp f.point_surjective|>
  id_comp f := by ext; rfl
  comp_id f := by ext; rfl
  assoc f g h := by ext; rfl

instance {X Y : Con} : CoeFun (X --> Y) (fun _ => X.Obj -> Y.Obj) where
  coe f := f.toFun

def Con.asFunctor {X Y : Con} (f : X --> Y) : X.Obj ==> Y.Obj where
  obj := f
  map g := homOfLE (f.monotone (leOfHom g))

abbrev CoCon := Opposite Con

def Tra : Con ==> Type where
  obj X := X.Obj
  map f := f.toFun

/-- A folio is stored by the least finite presentation of its eventually
constant spine functor. `length` is its cardinality: page `length - 1` is
the final genuine page and every later spine page repeats it. -/
structure Folio where
  length : Nat
  positive : 0 < length
  core : Functor.{0, 0} (Fin length) CoCon
  originEquiv : (core.obj <|0, positive|>).unop.Obj Equiv Unit

def Folio.paddedIndex (W : Folio) (n : Nat) : Fin W.length :=
  <|min n (W.length - 1), lt_of_le_of_lt (min_le_right _ _)
    (Nat.sub_lt W.positive (by omega))|>

theorem Folio.paddedIndex_mono (W : Folio) :
    Monotone W.paddedIndex := by
  intro a b hab
  exact min_le_min hab le_rfl

@[simp]
theorem Folio.paddedIndex_fin (W : Folio) (m : Fin W.length) :
    W.paddedIndex m.1 = m := by
  apply Fin.ext
  simp only [Folio.paddedIndex]
  exact Nat.min_eq_left (by omega)

def Folio.spineBase (W : Folio) : Nat ==> Fin W.length where
  obj n := W.paddedIndex n
  map f := homOfLE (W.paddedIndex_mono (leOfHom f))
  map_id _ := Subsingleton.elim _ _
  map_comp _ _ := Subsingleton.elim _ _

```

```
/-- The actual functor on the spine denoted by the finite presentation. -/
def Folio.F (W : Folio) : Nat ==> CoCon := W.spineBase then W.core

/-- The page functor on the opposite spine. -/
def Folio.spineH (W : Folio) : Natop ==> Type := W.F.leftOp then Tra

def Folio.H (W : Folio) : (Fin W.length)op ==> Type := W.core.leftOp then Tra

def Folio.E (W : Folio) : Type 0 := W.H.Elements

instance (W : Folio) : Category.{0} W.E := categoryOfElements W.H

instance (W : Folio) (m : (Fin W.length)op) : LinearOrder (W.H.obj m) :=
  (W.core.obj m.unop).unop.linearOrder

instance (W : Folio) (x y : W.E) : Subsingleton (x --> y) where
  allEq f g := CategoryOfElements.ext W.H f g (Subsingleton.elim _ _)

def Folio.originIndex (W : Folio) : Fin W.length := <|0, W.positive|>
def Folio.originBase (W : Folio) : (Fin W.length)op := op W.originIndex
def Folio.originValue (W : Folio) : W.H.obj W.originBase :=
  W.originEquiv.symm ()
def Folio.originElement (W : Folio) : W.E := <|W.originBase, W.originValue|>

def Folio.lastIndex (W : Folio) : Fin W.length :=
  <|W.length - 1, Nat.sub_lt W.positive (by omega)|>
def Folio.lastBase (W : Folio) : (Fin W.length)op := op W.lastIndex

theorem Folio.origin_unique (W : Folio) (x : W.H.obj W.originBase) :
    x = W.originValue := by
  apply W.originEquiv.injective
  exact Subsingleton.elim _ _

def Folio.baseToOrigin (W : Folio) (m : (Fin W.length)op) :
    m --> W.originBase := by
  letI : NeZero W.length := <|Nat.ne_of_gt W.positive|>
  exact (homOfLE (show W.originIndex <= m.unop by
    change (0 : Fin W.length) <= m.unop
    exact bot_le)).op

def Folio.toOrigin (W : Folio) (x : W.E) : x --> W.originElement :=
  CategoryOfElements.homMk x W.originElement (W.baseToOrigin x.1)
    (W.origin_unique _)

/-! The tall presentation is kept internal.  A folio's listed cardinality is
only a finite presentation of its genuinely `Nat`-indexed spine. -/

def Folio.TallE (W : Folio) : Type := W.spineH.Elements

instance (W : Folio) : Category W.TallE := categoryOfElements W.spineH

instance (W : Folio) (x y : W.TallE) : Subsingleton (x --> y) where
  allEq f g := CategoryOfElements.ext W.spineH f g (Subsingleton.elim _ _)

/-- Collapse a tall occurrence to the coherent finite representative used for
storage.  This is an implementation map, not the page space of the atlas. -/
def Folio.collapseElements (W : Folio) : W.TallE ==> W.E where
  obj x := <|op (W.paddedIndex x.1.unop), x.2|>
  map {x y} f := CategoryOfElements.homMk _ _
    (W.spineBase.map f.val.unop).op f.property
  map_id _ := by
    apply CategoryOfElements.ext
    apply Subsingleton.elim
  map_comp _ _ := by
    apply CategoryOfElements.ext
    apply Subsingleton.elim

/-- Include a stored cell at its actual page into the full spine. -/
def Folio.includeElements (W : Folio) : W.E ==> W.TallE where
  obj x := <|op x.1.unop.1,
    W.H.map (eqToHom
      (congrArg op (W.paddedIndex_fin x.1.unop).symm)) x.2|>
  map {x y} f := by
```

```
    rcases x with <|<|m|>, k|>
    rcases y with <|<|n|>, l|>
    have hnm : n.1 <= m.1 := leOfHom f.val.unop
    refine CategoryOfElements.homMk _ _ (homOfLE hnm).op ?_
    simp only [Folio.spineH, Folio.F, Folio.spineBase]
    change W.H.map _ (W.H.map _ k) = W.H.map _ l
    rw [<- FunctorToTypes.map_comp_apply,
      show _ >> _ = f.val >> _ from Subsingleton.elim _ _]
    rw [FunctorToTypes.map_comp_apply, f.property]
  map_id _ := by
    apply CategoryOfElements.ext
    apply Subsingleton.elim
  map_comp _ _ := by
    apply CategoryOfElements.ext
    apply Subsingleton.elim

@[simp]
theorem Folio.collapse_include (W : Folio) :
    W.includeElements then W.collapseElements = Unit W.E := by
  exact CategoryTheory.Functor.ext
    (F := W.includeElements then W.collapseElements) (G := Functor.id W.E)
    (fun x => by
      rcases x with <|<|m|>, k|>
      refine Functor.Elements.ext _ _ (congrArg op (W.paddedIndex_fin m)) ?_
      simp [Folio.includeElements, Folio.collapseElements])

@[simp]
theorem Folio.collapse_include_obj (W : Folio) (x : W.E) :
    W.collapseElements.obj (W.includeElements.obj x) = x := by
  exact Functor.congr_obj W.collapse_include x

/-- The finite presentation is a retract of the full spine, but not conversely:
collapsing the first repeated page and including it again changes its page
index.  This records the precise obstruction to treating the two atlas bases
as interchangeable through the evident inclusion/collapse comparison. -/
theorem Folio.include_collapse_ne_id (W : Folio) :
    W.collapseElements then W.includeElements != Functor.id W.TallE := by
  intro h
  letI : NeZero W.length := <|Nat.ne_of_gt W.positive|>
  let q : (W.core.obj W.lastIndex).unop -->
      (W.core.obj W.originIndex).unop :=
    (W.core.map (homOfLE (Fin.zero_le W.lastIndex))).unop
  let k : (W.core.obj W.lastIndex).unop.Obj :=
    Classical.choose (q.point_surjective W.originValue)
  let x : W.TallE := by
    refine <|op W.length, ?_|>
    change (W.core.obj (W.paddedIndex W.length)).unop.Obj
    have hp : W.paddedIndex W.length = W.lastIndex := by
      apply Fin.ext
      simp [Folio.paddedIndex, Folio.lastIndex]
    exact hp.symm transport k
  have hx := congrArg
    (fun F : CategoryTheory.Functor W.TallE W.TallE => (F.obj x).1.unop) h
  have hbad : W.length - 1 = W.length := by
    simpa [x, Folio.collapseElements, Folio.includeElements,
      Folio.paddedIndex] using hx
  exact (ne_of_lt (Nat.sub_lt W.positive (by omega))) hbad

structure Pag where
  folio : Folio

def Pag.H (W : Pag) : (Fin W.folio.length)op ==> Type 0 := W.folio.H
def Pag.E (W : Pag) : Type 0 := W.folio.E

instance (W : Pag) : Category.{0} W.E := categoryOfElements W.H

instance : Category.{0} Pag where
  Hom X Y := X.E ==> Y.E
  id X := Unit X.E
  comp F G := F then G
  id_comp _ := rfl
  comp_id _ := rfl
  assoc _ _ _ := rfl
```

```
359
360 /-- Internally, an atlas is stored by its least finite presentation.  The
361 `Folio.F` construction repeats its final page along the rest of the spine, so
362 the stabilization and morphism-tail clauses in the extracted definition are
363 enforced by construction. -/
364
365 def Pag.cell (P : Pag) (m : (Fin P.folio.length)op) (k : P.H.obj m) : P.E := <|m, k|>
366
367 def Pag.cellToOrigin (P : Pag) (m : (Fin P.folio.length)op)
368     (k : P.H.obj m) : P.cell m k --> P.folio.originElement :=
369   P.folio.toOrigin (P.cell m k)
370
371 def IsPagewiseDisjoint (P : Pag) (G : P.E ==> DomIns) : Prop :=
372   forall  (m : (Fin P.folio.length)op) (i j : P.H.obj m), i != j ->
373     forall  (x : G.obj (P.cell m i)) (y : G.obj (P.cell m j)),
374       G.map (P.cellToOrigin m i) x != G.map (P.cellToOrigin m j) y
375
376 structure Atl where
377   P : Pag
378   G : P.E ==> DomIns
379   disjoint : IsPagewiseDisjoint P G
380
381 def Atl.H (X : Atl) : (Fin X.P.folio.length)op ==> Type 0 := X.P.H
382 def Atl.E (X : Atl) : Type 0 := X.P.E
383
384 /-- The `n`th page of an atlas on its genuine infinite spine. -/
385 def Atl.page (X : Atl) (n : Nat) : Type := X.P.folio.spineH.obj (op n)
386
387 /-- A cell of the `n`th page, retaining its position on the infinite spine. -/
388 def Atl.pageCell (X : Atl) (n : Nat) (k : X.page n) : X.P.folio.TallE :=
389   <|op n, k|>
390
391 instance (X : Atl) : Category.{0} X.E := categoryOfElements X.H
392
393 structure AtlHom (X Y : Atl) where
394   P : X.E ==> Y.E
395   A : X.G --> P then Y.G
396
397 def AtlHom.identity (X : Atl) : AtlHom X X where
398   P := Unit X.E
399   A := Id X.G
400
401 def AtlHom.comp {X Y Z : Atl} (f : AtlHom X Y) (g : AtlHom Y Z) : AtlHom X Z where
402   P := f.P then g.P
403   A := f.A >> whiskerLeft f.P g.A
404
405 @[ext]
406 theorem AtlHom.ext {X Y : Atl} (f g : AtlHom X Y)
407     (hP : f.P = g.P) (hA : HEq f.A g.A) : f = g := by
408   cases f
409   cases g
410   simp_all
411
412 /-- Heterogeneous extensionality for natural transformations whose target
413 functors have been identified.  This keeps dependent transports localized. -/
414 theorem NatTrans.hext_right {C D : Type*} [Category C] [Category D]
415     {F G H : C ==> D} (alpha : F --> G) (beta : F --> H) (h : G = H)
416     (happ : forall  X, HEq (alpha.app X) (beta.app X)) : HEq alpha beta := by
417   cases h
418   apply heq_of_eq
419   ext X
420   exact eq_of_heq (happ X)
421
422 /-- Transporting the codomain of an embedding along an equality of objects is
423 heterogeneously equal to the original embedding.  This is the small coherence
424 fact needed whenever stability identifies the image of an origin with the
425 target origin. -/
426 theorem embedding_comp_map_eqToHom_heq {C : Type u} [Category C]
427     (G : CategoryTheory.Functor C DomIns) {a b : C} (h : a = b) {X : DomIns}
428     (e : X --> G.obj a) : HEq (e >> G.map (eqToHom h)) e := by
429   cases h
430   simp
431
```

```
instance : Category.{0} Atl where
  Hom := AtlHom
  id := AtlHom.identity
  comp := AtlHom.comp
  id_comp f := by
    apply AtlHom.ext
    . rfl
    . apply heq_of_eq
      ext x
      simp [AtlHom.comp, AtlHom.identity]
  comp_id f := by
    apply AtlHom.ext
    . rfl
    . apply heq_of_eq
      ext x
      simp [AtlHom.comp, AtlHom.identity]
  assoc f g h := by
    apply AtlHom.ext
    . rfl
    . apply heq_of_eq
      ext x
      simp [AtlHom.comp]

def Folio.commonBase (W : Folio) (m n : (Fin W.length)op) :
    (Fin W.length)op := op (min m.unop n.unop)

def Folio.toCommonLeft (W : Folio) (m n : (Fin W.length)op) :
    m --> W.commonBase m n := (homOfLE (min_le_left _ _)).op

def Folio.toCommonRight (W : Folio) (m n : (Fin W.length)op) :
    n --> W.commonBase m n := (homOfLE (min_le_right _ _)).op

def storedElementLT (X : Atl) (x y : X.E) : Prop :=
  let m := X.P.folio.commonBase x.1 y.1
  letI : LinearOrder (X.H.obj m) :=
    (X.P.folio.core.obj m.unop).unop.linearOrder
  X.H.map (X.P.folio.toCommonLeft x.1 y.1) x.2 <
  X.H.map (X.P.folio.toCommonRight x.1 y.1) y.2

def storedExtent (A : Atl) : DomIns := A.G.obj A.P.folio.originElement

def StoredTerritoryIndex (A : Atl) : Type := A.H.obj A.P.folio.lastBase

def storedTerritory (A : Atl) (k : StoredTerritoryIndex A) : DomIns :=
  A.G.obj (A.P.cell A.P.folio.lastBase k)

def storedOriginImage (X : Atl) (x : X.E) (t : X.G.obj x) : storedExtent X :=
  X.G.map (X.P.folio.toOrigin x) t

def storedCovered (X : Atl) (x : X.E) (t : X.G.obj x) : Prop :=
  exists  (k : X.H.obj X.P.folio.lastBase)
      (l : X.G.obj (X.P.cell X.P.folio.lastBase k)),
    storedOriginImage X x t =
      storedOriginImage X (X.P.cell X.P.folio.lastBase k) l

/-! `TallAtlas` is the internal, genuinely infinite-spine presentation.  An
`Atl` supplies the coherent page data by pulling its finite storage back along
`collapseElements`.  This distinction deliberately stays out of the extracted
mathematical definitions. -/

structure TallAtlas where
  H : Natop ==> Type
  originValue : H.obj (op 0)
  originUnique : forall  x : H.obj (op 0), x = originValue
  G : H.Elements ==> DomIns
  cellLT : H.Elements -> H.Elements -> Prop
  coveredExtent : Set (G.obj <|op 0, originValue|>)

def TallAtlas.E (X : TallAtlas) : Type := X.H.Elements

instance (X : TallAtlas) : Category X.E := categoryOfElements X.H

instance (X : TallAtlas) (x y : X.E) : Subsingleton (x --> y) where
```

```
  allEq f g := CategoryOfElements.ext X.H f g (Subsingleton.elim _ _)

def Atl.tallOriginValue (X : Atl) : X.P.folio.spineH.obj (op 0) := by
  simpa [Folio.spineH, Folio.F, Folio.spineBase, Folio.paddedIndex,
    Folio.originIndex] using X.P.folio.originValue

def Atl.tall (X : Atl) : TallAtlas where
  H := X.P.folio.spineH
  originValue := X.tallOriginValue
  originUnique x := by
    let e : X.P.folio.spineH.obj (op 0) ≃ Unit := by
      simpa [Folio.spineH, Folio.F, Folio.spineBase, Folio.paddedIndex,
        Folio.originIndex] using X.P.folio.originEquiv
    apply e.injective
    exact Subsingleton.elim _ _
  G := X.P.folio.collapseElements then X.G
  cellLT x y := storedElementLT X
    (X.P.folio.collapseElements.obj x)
    (X.P.folio.collapseElements.obj y)
  coveredExtent t := storedCovered X
    (X.P.folio.collapseElements.obj
      <|op 0, X.tallOriginValue|>) t

def TallAtlas.originElement (X : TallAtlas) : X.E :=
  <|op 0, X.originValue|>

def TallAtlas.toOrigin (X : TallAtlas) (x : X.E) : x --> X.originElement :=
  CategoryOfElements.homMk x X.originElement
    (homOfLE (Nat.zero_le x.1.unop)).op (X.originUnique _)

def TallAtlas.covered (X : TallAtlas) (x : X.E) (t : X.G.obj x) : Prop :=
  X.coveredExtent (X.G.map (X.toOrigin x) t)

def TallAtlas.extent (X : TallAtlas) : DomIns := X.G.obj X.originElement

structure TallAtlasHom (X Y : TallAtlas) where
  P : X.E ==> Y.E
  A : X.G --> P then Y.G

def TallAtlasHom.identity (X : TallAtlas) : TallAtlasHom X X where
  P := Unit X.E
  A := Id X.G

def TallAtlasHom.comp {X Y Z : TallAtlas}
    (f : TallAtlasHom X Y) (g : TallAtlasHom Y Z) : TallAtlasHom X Z where
  P := f.P then g.P
  A := f.A >> whiskerLeft f.P g.A

@[ext]
theorem TallAtlasHom.ext {X Y : TallAtlas} (f g : TallAtlasHom X Y)
    (hP : f.P = g.P) (hA : HEq f.A g.A) : f = g := by
  cases f
  cases g
  simp_all

instance : Category TallAtlas where
  Hom := TallAtlasHom
  id := TallAtlasHom.identity
  comp := TallAtlasHom.comp
  id_comp f := by
    apply TallAtlasHom.ext
    . rfl
    . apply heq_of_eq
      ext x
      simp [TallAtlasHom.comp, TallAtlasHom.identity]
  comp_id f := by
    apply TallAtlasHom.ext
    . rfl
    . apply heq_of_eq
      ext x
      simp [TallAtlasHom.comp, TallAtlasHom.identity]
  assoc f g h := by
    apply TallAtlasHom.ext
```

```
    . rfl
    . apply heq_of_eq
      ext x
      simp [TallAtlasHom.comp]

/-! An atlas arrow's action on the infinite presentation is derived from its
finite pagination and data transformation.  The full spine is first collapsed
to the coherent stored representative, transformed by `P` and `A`, and then
included at the resulting stored page.  Thus no independent spine action is
part of an atlas morphism. -/

def AtlHom.tallP {X Y : Atl} (f : X --> Y) :
    X.tall.E ==> Y.tall.E :=
  X.P.folio.collapseElements then f.P then Y.P.folio.includeElements

def AtlHom.tallA {X Y : Atl} (f : X --> Y) :
    X.tall.G --> f.tallP then Y.tall.G where
  app x := f.A.app (X.P.folio.collapseElements.obj x) >>
    Y.G.map (eqToHom (Y.P.folio.collapse_include_obj
      (f.P.obj (X.P.folio.collapseElements.obj x))).symm)
  naturality := by
    intro x y q
    simp only [AtlHom.tallP, Atl.tall, Functor.comp_obj, Functor.comp_map]
    rw [<- Category.assoc, f.A.naturality, Category.assoc]
    simp only [Functor.comp_map]
    have he :
        Y.G.map (f.P.map (X.P.folio.collapseElements.map q)) >>
          Y.G.map (eqToHom (Y.P.folio.collapse_include_obj _).symm) =
        Y.G.map (eqToHom (Y.P.folio.collapse_include_obj _).symm) >>
          Y.G.map (Y.P.folio.collapseElements.map
            (Y.P.folio.includeElements.map
              (f.P.map (X.P.folio.collapseElements.map q)))) := by
      rw [<- Y.G.map_comp, <- Y.G.map_comp]
      congr 1
    rw [he]
    simp only [Category.assoc]

def AtlHom.tall {X Y : Atl} (f : X --> Y) : X.tall --> Y.tall where
  P := f.tallP
  A := f.tallA

/-- The coherent identity of an infinite presentation.  It identifies every
repeated occurrence with the stored representative. -/
def Atl.coherence (X : Atl) : X.tall --> X.tall :=
  (AtlHom.identity X).tall

/-- Deriving the spine action commutes strictly with composition. -/
theorem AtlHom.tall_comp {X Y Z : Atl} (f : X --> Y) (g : Y --> Z) :
    (f >> g).tall = f.tall >> g.tall := by
  have hP : (f >> g).tall.P = (f.tall >> g.tall).P := by
    exact CategoryTheory.Functor.ext
      (F := (f >> g).tall.P) (G := (f.tall >> g.tall).P)
      (fun x => by
        change Z.P.folio.includeElements.obj
            (g.P.obj (f.P.obj (X.P.folio.collapseElements.obj x))) =
          Z.P.folio.includeElements.obj
            (g.P.obj (Y.P.folio.collapseElements.obj
              (Y.P.folio.includeElements.obj
                (f.P.obj (X.P.folio.collapseElements.obj x)))))
        rw [Y.P.folio.collapse_include_obj])
  apply TallAtlasHom.ext _ _ hP
  apply NatTrans.hext_right _ _
    (congrArg (fun P => P then Z.tall.G) hP)
  intro x
  let x_0 := X.P.folio.collapseElements.obj x
  let y_0 := f.P.obj x_0
  let eY : y_0 = Y.P.folio.collapseElements.obj
      (Y.P.folio.includeElements.obj y_0) :=
    (Y.P.folio.collapse_include_obj y_0).symm
  let z_0 := g.P.obj y_0
  let eZ : z_0 = Z.P.folio.collapseElements.obj
      (Z.P.folio.includeElements.obj z_0) :=
    (Z.P.folio.collapse_include_obj z_0).symm
```

```
  let y_1 := Y.P.folio.collapseElements.obj
    (Y.P.folio.includeElements.obj y_0)
  let z_1 := g.P.obj y_1
  let eZ_1 : z_1 = Z.P.folio.collapseElements.obj
      (Z.P.folio.includeElements.obj z_1) :=
    (Z.P.folio.collapse_include_obj z_1).symm
  let eR : z_0 = Z.P.folio.collapseElements.obj
      (Z.P.folio.includeElements.obj z_1) :=
    (congrArg g.P.obj eY).trans eZ_1
  change HEq
    ((AtlHom.comp f g).tallA.app x)
    ((TallAtlasHom.comp f.tall g.tall).A.app x)
  dsimp [AtlHom.tall, AtlHom.tallA, AtlHom.tallP, AtlHom.comp,
    TallAtlasHom.comp]
  rw [Category.assoc (f.A.app x_0) (Y.G.map (eqToHom eY))
    (g.A.app y_1 >> Z.G.map (eqToHom eZ_1))]
  rw [g.A.naturality_assoc (eqToHom eY)]
  change HEq
    ((f.A.app x_0 >> g.A.app y_0) >> Z.G.map (eqToHom eZ))
    (((f.A.app x_0 >> g.A.app y_0) >> Z.G.map (g.P.map (eqToHom eY))) >>
      Z.G.map (eqToHom eZ_1))
  have hrArrow : g.P.map (eqToHom eY) >> eqToHom eZ_1 = eqToHom eR :=
    CategoryOfElements.ext Z.H _ _ (Subsingleton.elim _ _)
  have hright : HEq
      (((f.A.app x_0 >> g.A.app y_0) >> Z.G.map (g.P.map (eqToHom eY))) >>
        Z.G.map (eqToHom eZ_1))
      (f.A.app x_0 >> g.A.app y_0) := by
    rw [Category.assoc, <- Z.G.map_comp, hrArrow]
    exact embedding_comp_map_eqToHom_heq Z.G eR _
  exact (embedding_comp_map_eqToHom_heq Z.G eZ
    (f.A.app x_0 >> g.A.app y_0)).trans hright.symm

theorem Atl.coherence_idempotent (X : Atl) :
    X.coherence >> X.coherence = X.coherence := by
  have h := AtlHom.tall_comp (Id X) (Id X)
  simpa [Atl.coherence] using h.symm

theorem Atl.coherence_ne_identity (X : Atl) :
    X.coherence != Id X.tall := by
  intro h
  apply X.P.folio.include_collapse_ne_id
  simpa [Atl.coherence, AtlHom.tall, AtlHom.tallP, AtlHom.identity,
    TallAtlasHom.identity] using congrArg TallAtlasHom.P h

theorem AtlHom.coherence_left {X Y : Atl} (f : X --> Y) :
    X.coherence >> f.tall = f.tall := by
  have h := AtlHom.tall_comp (Id X) f
  simpa [Atl.coherence] using h.symm

theorem AtlHom.coherence_right {X Y : Atl} (f : X --> Y) :
    f.tall >> Y.coherence = f.tall := by
  have h := AtlHom.tall_comp f (Id Y)
  simpa [Atl.coherence] using h.symm

def cardinality (A : Atl) : Nat := A.P.folio.length

def extent (A : Atl) : DomIns := storedExtent A

def TerritoryIndex (A : Atl) : Type := StoredTerritoryIndex A

def territory (A : Atl) (k : TerritoryIndex A) : DomIns :=
  storedTerritory A k

def region (A : Atl) (n : TerritoryIndex A) : DomIns := territory A n

def IsTransposal : MorphismProperty Atl :=
  fun _ _ F => Function.Injective F.P.obj

instance : IsTransposal.IsMultiplicative where
  id_mem _ := Function.injective_id
  comp_mem _ _ hf hg := hg.comp hf

abbrev AtlTrap := WideSubcategory IsTransposal
```

```

def AtlTrapInc : AtlTrap ==> Atl := wideSubcategoryInclusion IsTransposal

def elementLT (X : Atl) (x y : X.E) : Prop :=
  let m := X.P.folio.commonBase x.1 y.1
  letI : LinearOrder (X.H.obj m) :=
    (X.P.folio.core.obj m.unop).unop.linearOrder
  X.H.map (X.P.folio.toCommonLeft x.1 y.1) x.2 <
  X.H.map (X.P.folio.toCommonRight x.1 y.1) y.2

/-- The image of a datum in the extent. -/
def originImage (X : Atl) (x : X.E) (t : X.G.obj x) : extent X :=
  storedOriginImage X x t

/-- A datum is covered when its image in the extent comes from a final region. -/
def Covered (X : Atl) (x : X.E) (t : X.G.obj x) : Prop :=
  storedCovered X x t

theorem covered_region (X : Atl) (k : TerritoryIndex X) (l : territory X k) :
    Covered X (X.P.cell X.P.folio.lastBase k) l :=
  <|k, l, rfl|>

/-- The order and coverage conditions for atlas traversals. -/
def IsTraversal : MorphismProperty Atl := fun X Y F =>
  Function.Injective F.P.obj  /\
  (forall  x y, elementLT _ x y -> elementLT _ (F.P.obj x) (F.P.obj y))  /\
  (forall  x (t : X.G.obj x), Covered X x t ->
    Covered Y (F.P.obj x) (F.A.app x t))

instance : IsTraversal.IsMultiplicative where
  id_mem X := by
    refine <|Function.injective_id, fun _ _ h => h, ?_|>
    intro x t h
    simpa [AtlHom.identity] using h
  comp_mem f g hf hg := by
    refine <|hg.1.comp hf.1, fun x y h => hg.2.1 _ _ (hf.2.1 _ _ h), ?_|>
    intro x t h
    simpa [AtlHom.comp] using hg.2.2 (f.P.obj x) (f.A.app x t) (hf.2.2 x t h)

abbrev AtlTrav := WideSubcategory IsTraversal

def AtlTravInc : AtlTrav ==> Atl := wideSubcategoryInclusion IsTraversal

def AtlTravToAtlTrap : AtlTrav ==> AtlTrap where
  obj X := WideSubcategory.mk X.obj
  map f := <|f.1, f.2.1|>

def IsStableTraversal : MorphismProperty AtlTrav := fun X Y F =>
  F.1.P.obj X.obj.P.folio.originElement = Y.obj.P.folio.originElement

instance : IsStableTraversal.IsMultiplicative where
  id_mem _ := rfl
  comp_mem f g hf hg := by
    change g.1.P.obj (f.1.P.obj _) = _
    rw [hf, hg]

abbrev StaAtlTrav := WideSubcategory IsStableTraversal

def StaAtlTravToAtlTrav : StaAtlTrav ==> AtlTrav :=
  wideSubcategoryInclusion IsStableTraversal

def StaAtlTravInc : StaAtlTrav ==> Atl := StaAtlTravToAtlTrav then AtlTravInc

/-! `StableAtlasFamily` implements atlas federations of stable atlases.
The empty family is the empty atlas operation, and concatenation retains all
components without choosing representatives or identifying their data.  Most
importantly, every component arrow is an arrow of `StaAtlTrav`, so stability is
checked by Lean at the boundary of the horizontal construction. -/

structure StableAtlasFamily where
  Index : Type
  countableIndex : Countable Index
  component : Index -> StaAtlTrav
```

```
797
798 attribute [instance] StableAtlasFamily.countableIndex
799
800 structure StableAtlasFamilyHom (X Y : StableAtlasFamily) where
801   index : X.Index -> Y.Index
802   component : forall  i, X.component i --> Y.component (index i)
803
804 /-- Each component of an atlas-federation morphism is stable by construction. -/
805 theorem StableAtlasFamilyHom.component_stable {X Y : StableAtlasFamily}
806     (f : StableAtlasFamilyHom X Y) (i : X.Index) :
807     IsStableTraversal (f.component i).1 :=
808   (f.component i).2
809
810 @[ext]
811 theorem StableAtlasFamilyHom.ext {X Y : StableAtlasFamily}
812     (f g : StableAtlasFamilyHom X Y) (hi : f.index = g.index)
813     (hc : forall  i, HEq (f.component i) (g.component i)) : f = g := by
814   cases f
815   cases g
816   cases hi
817   congr
818   funext i
819   exact eq_of_heq (hc i)
820
821 def StableAtlasFamilyHom.identity (X : StableAtlasFamily) :
822     StableAtlasFamilyHom X X where
823   index := id
824   component := fun _ => Id _
825
826 def StableAtlasFamilyHom.comp {X Y Z : StableAtlasFamily}
827     (f : StableAtlasFamilyHom X Y) (g : StableAtlasFamilyHom Y Z) :
828     StableAtlasFamilyHom X Z where
829   index := g.index  o  f.index
830   component := fun i => f.component i >> g.component (f.index i)
831
832 instance : Category StableAtlasFamily where
833   Hom := StableAtlasFamilyHom
834   id := StableAtlasFamilyHom.identity
835   comp := StableAtlasFamilyHom.comp
836   id_comp f := by
837     apply StableAtlasFamilyHom.ext
838     . rfl
839     . intro i
840       exact heq_of_eq (Category.id_comp _)
841   comp_id f := by
842     apply StableAtlasFamilyHom.ext
843     . rfl
844     . intro i
845       exact heq_of_eq (Category.comp_id _)
846   assoc f g h := by
847     apply StableAtlasFamilyHom.ext
848     . rfl
849     . intro i
850       exact heq_of_eq (Category.assoc _ _ _)
851
852 @[simp]
853 theorem StableAtlasFamilyHom.component_id (X : StableAtlasFamily)
854     (i : X.Index) :
855     (Id X : X --> X).component i = Id (X.component i) := rfl
856
857 @[simp]
858 theorem StableAtlasFamilyHom.component_comp {X Y Z : StableAtlasFamily}
859     (f : X --> Y) (g : Y --> Z) (i : X.Index) :
860     (f >> g).component i = f.component i >> g.component (f.index i) := rfl
861
862 /-- An atlas as a single-component atlas federation. -/
863 def stableAtlasAtom (X : StaAtlTrav) : StableAtlasFamily where
864   Index := Unit
865   countableIndex := inferInstance
866   component := fun _ => X
867
868 /-- The empty atlas federation has no components. -/
869 def stableAtlasUnit : StableAtlasFamily where
```

```
  Index := Empty
  countableIndex := inferInstance
  component := fun i => nomatch i

/-- Federation merge is disjoint tagged retention of components,
not a categorical coproduct of atlases. -/
def stableAtlasMerge (X Y : StableAtlasFamily) : StableAtlasFamily where
  Index := Sum X.Index Y.Index
  countableIndex := inferInstance
  component := Sum.elim X.component Y.component

def stableAtlasMergeHom {X_1 X_2 Y_1 Y_2 : StableAtlasFamily}
    (f : X_1 --> Y_1) (g : X_2 --> Y_2) :
    stableAtlasMerge X_1 X_2 --> stableAtlasMerge Y_1 Y_2 where
  index := Sum.map f.index g.index
  component
    | .inl i => f.component i
    | .inr j => g.component j

/-- Horizontal sum on atlas federations. -/
def StableAtlHorSum : StableAtlasFamily * StableAtlasFamily ==>
    StableAtlasFamily where
  obj X := stableAtlasMerge X.1 X.2
  map f := stableAtlasMergeHom f.1 f.2
  map_id X := by
    apply StableAtlasFamilyHom.ext
    . funext i
      cases i <;> rfl
    . intro i
      cases i <;> exact HEq.rfl
  map_comp f g := by
    apply StableAtlasFamilyHom.ext
    . funext i
      cases i <;> rfl
    . intro i
      cases i <;> exact HEq.rfl

def stableAtlasAssociatorHom (X Y Z : StableAtlasFamily) :
    stableAtlasMerge (stableAtlasMerge X Y) Z -->
      stableAtlasMerge X (stableAtlasMerge Y Z) where
  index
    | .inl (.inl i) => .inl i
    | .inl (.inr j) => .inr (.inl j)
    | .inr k => .inr (.inr k)
  component
    | .inl (.inl i) => Id (X.component i)
    | .inl (.inr j) => Id (Y.component j)
    | .inr k => Id (Z.component k)

def stableAtlasAssociatorInv (X Y Z : StableAtlasFamily) :
    stableAtlasMerge X (stableAtlasMerge Y Z) -->
      stableAtlasMerge (stableAtlasMerge X Y) Z where
  index
    | .inl i => .inl (.inl i)
    | .inr (.inl j) => .inl (.inr j)
    | .inr (.inr k) => .inr k
  component
    | .inl i => Id (X.component i)
    | .inr (.inl j) => Id (Y.component j)
    | .inr (.inr k) => Id (Z.component k)

def stableAtlasAssociator (X Y Z : StableAtlasFamily) :
    stableAtlasMerge (stableAtlasMerge X Y) Z Iso
      stableAtlasMerge X (stableAtlasMerge Y Z) where
  hom := stableAtlasAssociatorHom X Y Z
  inv := stableAtlasAssociatorInv X Y Z
  hom_inv_id := by
    apply StableAtlasFamilyHom.ext
    . funext i
      rcases i with (i | j) | k <;> rfl
    . intro i
      rcases i with (i | j) | k <;>
        simp [stableAtlasAssociatorHom, stableAtlasAssociatorInv,
```

```
          stableAtlasMerge]
  inv_hom_id := by
    apply StableAtlasFamilyHom.ext
    . funext i
      rcases i with i | (j | k) <;> rfl
    . intro i
      rcases i with i | (j | k) <;>
        simp [stableAtlasAssociatorHom, stableAtlasAssociatorInv,
          stableAtlasMerge]

def stableAtlasLeftUnitorHom (X : StableAtlasFamily) :
    stableAtlasMerge stableAtlasUnit X --> X where
  index
    | .inr i => i
  component
    | .inr i => Id (X.component i)

def stableAtlasLeftUnitorInv (X : StableAtlasFamily) :
    X --> stableAtlasMerge stableAtlasUnit X where
  index i := .inr i
  component i := Id (X.component i)

def stableAtlasLeftUnitor (X : StableAtlasFamily) :
    stableAtlasMerge stableAtlasUnit X Iso X where
  hom := stableAtlasLeftUnitorHom X
  inv := stableAtlasLeftUnitorInv X
  hom_inv_id := by
    apply StableAtlasFamilyHom.ext
    . funext i
      rcases i with i | i
      . exact i.elim
      . rfl
    . intro i
      rcases i with i | i
      . exact i.elim
      . simp [stableAtlasLeftUnitorHom, stableAtlasLeftUnitorInv,
          stableAtlasMerge]
  inv_hom_id := by
    apply StableAtlasFamilyHom.ext
    . rfl
    . intro i
      simp [stableAtlasLeftUnitorHom, stableAtlasLeftUnitorInv,
        stableAtlasMerge]

def stableAtlasRightUnitorHom (X : StableAtlasFamily) :
    stableAtlasMerge X stableAtlasUnit --> X where
  index
    | .inl i => i
  component
    | .inl i => Id (X.component i)

def stableAtlasRightUnitorInv (X : StableAtlasFamily) :
    X --> stableAtlasMerge X stableAtlasUnit where
  index i := .inl i
  component i := Id (X.component i)

def stableAtlasRightUnitor (X : StableAtlasFamily) :
    stableAtlasMerge X stableAtlasUnit Iso X where
  hom := stableAtlasRightUnitorHom X
  inv := stableAtlasRightUnitorInv X
  hom_inv_id := by
    apply StableAtlasFamilyHom.ext
    . funext i
      rcases i with i | i
      . rfl
      . exact i.elim
    . intro i
      rcases i with i | i
      . simp [stableAtlasRightUnitorHom, stableAtlasRightUnitorInv,
          stableAtlasMerge]
      . exact i.elim
  inv_hom_id := by
    apply StableAtlasFamilyHom.ext
```

```
    . rfl
    . intro i
      simp [stableAtlasRightUnitorHom, stableAtlasRightUnitorInv,
        stableAtlasMerge]

def stableAtlasBraidingHom (X Y : StableAtlasFamily) :
    stableAtlasMerge X Y --> stableAtlasMerge Y X where
  index
    | .inl i => .inr i
    | .inr j => .inl j
  component
    | .inl i => Id (X.component i)
    | .inr j => Id (Y.component j)

def stableAtlasBraiding (X Y : StableAtlasFamily) :
    stableAtlasMerge X Y Iso stableAtlasMerge Y X where
  hom := stableAtlasBraidingHom X Y
  inv := stableAtlasBraidingHom Y X
  hom_inv_id := by
    apply StableAtlasFamilyHom.ext
    . funext i
      cases i <;> rfl
    . intro i
      cases i <;> simp [stableAtlasBraidingHom, stableAtlasMerge]
  inv_hom_id := by
    apply StableAtlasFamilyHom.ext
    . funext i
      cases i <;> rfl
    . intro i
      cases i <;> simp [stableAtlasBraidingHom, stableAtlasMerge]

instance stableAtlasMonoidalStruct : MonoidalCategoryStruct StableAtlasFamily where
  tensorObj := stableAtlasMerge
  tensorHom := stableAtlasMergeHom
  whiskerLeft X _ _ f := stableAtlasMergeHom (Id X) f
  whiskerRight f Y := stableAtlasMergeHom f (Id Y)
  tensorUnit := stableAtlasUnit
  associator := stableAtlasAssociator
  leftUnitor := stableAtlasLeftUnitor
  rightUnitor := stableAtlasRightUnitor

instance stableAtlasMonoidal : MonoidalCategory StableAtlasFamily :=
  MonoidalCategory.ofTensorHom
    (id_tensorHom_id := fun X Y => StableAtlHorSum.map_id (X, Y))
    (id_tensorHom := fun X {_ _} f => rfl)
    (tensorHom_id := fun {_ _} f Y => rfl)
    (tensorHom_comp_tensorHom := fun {X_1 Y_1 Z_1 X_2 Y_2 Z_2} f_1 f_2 g_1 g_2 => by
      apply StableAtlasFamilyHom.ext
      . funext i
        cases i <;> rfl
      . intro i
        cases i <;> simp [stableAtlasMonoidalStruct, stableAtlasMergeHom])
    (associator_naturality := fun {_ _ _ _ _ _} f_1 f_2 f_3 => by
      apply StableAtlasFamilyHom.ext
      . funext i
        rcases i with (i | j) | k <;> rfl
      . intro i
        rcases i with (i | j) | k <;>
          simp [stableAtlasMonoidalStruct, stableAtlasMergeHom,
            stableAtlasAssociator, stableAtlasAssociatorHom, stableAtlasMerge])
    (leftUnitor_naturality := fun {_ _} f => by
      apply StableAtlasFamilyHom.ext
      . funext i
        rcases i with i | i
        . exact i.elim
        . rfl
      . intro i
        rcases i with i | i
        . exact i.elim
        . simp [stableAtlasMonoidalStruct, stableAtlasMergeHom,
            stableAtlasLeftUnitor, stableAtlasLeftUnitorHom, stableAtlasMerge])
    (rightUnitor_naturality := fun {_ _} f => by
      apply StableAtlasFamilyHom.ext
```

```
      . funext i
        rcases i with i | i
        . rfl
        . exact i.elim
      . intro i
        rcases i with i | i
        . simp [stableAtlasMonoidalStruct, stableAtlasMergeHom,
            stableAtlasRightUnitor, stableAtlasRightUnitorHom, stableAtlasMerge]
        . exact i.elim)
    (pentagon := fun W X Y Z => by
      apply StableAtlasFamilyHom.ext
      . funext i
        rcases i with ((i | j) | k) | l <;> rfl
      . intro i
        rcases i with ((i | j) | k) | l <;>
          simp [stableAtlasMonoidalStruct, stableAtlasMergeHom,
            stableAtlasAssociator, stableAtlasAssociatorHom, stableAtlasMerge])
    (triangle := fun X Y => by
      apply StableAtlasFamilyHom.ext
      . funext i
        rcases i with (i | e) | j
        . rfl
        . exact e.elim
        . rfl
      . intro i
        rcases i with (i | e) | j
        . simp [stableAtlasMonoidalStruct, stableAtlasMergeHom,
            stableAtlasAssociator, stableAtlasAssociatorHom,
            stableAtlasRightUnitor, stableAtlasRightUnitorHom, stableAtlasMerge]
        . exact e.elim
        . simp [stableAtlasMonoidalStruct, stableAtlasMergeHom,
            stableAtlasAssociator, stableAtlasAssociatorHom,
            stableAtlasLeftUnitor, stableAtlasLeftUnitorHom, stableAtlasMerge])

@[simp]
theorem stableAtlas_tensorObj (X Y : StableAtlasFamily) :
    MonoidalCategoryStruct.tensorObj X Y = stableAtlasMerge X Y := rfl

@[simp]
theorem stableAtlas_tensorHom {X_1 Y_1 X_2 Y_2 : StableAtlasFamily}
    (f : X_1 --> Y_1) (g : X_2 --> Y_2) :
    MonoidalCategoryStruct.tensorHom f g = stableAtlasMergeHom f g := rfl

@[simp]
theorem stableAtlas_whiskerLeft (X : StableAtlasFamily)
    {Y_1 Y_2 : StableAtlasFamily} (f : Y_1 --> Y_2) :
    MonoidalCategoryStruct.whiskerLeft X f =
      stableAtlasMergeHom (Id X) f := rfl

@[simp]
theorem stableAtlas_whiskerRight {X_1 X_2 : StableAtlasFamily}
    (f : X_1 --> X_2) (Y : StableAtlasFamily) :
    MonoidalCategoryStruct.whiskerRight f Y =
      stableAtlasMergeHom f (Id Y) := rfl

@[simp]
theorem stableAtlas_associator (X Y Z : StableAtlasFamily) :
    MonoidalCategoryStruct.associator X Y Z =
      stableAtlasAssociator X Y Z := rfl

instance stableAtlasBraided : BraidedCategory StableAtlasFamily where
  braiding := stableAtlasBraiding
  braiding_naturality_right := fun X {_ _} f => by
    apply StableAtlasFamilyHom.ext
    . funext i
      cases i <;> rfl
    . intro i
      cases i <;>
        simp [stableAtlasMergeHom, stableAtlasBraiding,
          stableAtlasBraidingHom, stableAtlasMerge]
  braiding_naturality_left := fun {_ _} f Z => by
    apply StableAtlasFamilyHom.ext
    . funext i
```

```
      cases i <;> rfl
    . intro i
      cases i <;>
        simp [stableAtlasMergeHom, stableAtlasBraiding,
          stableAtlasBraidingHom, stableAtlasMerge]
  hexagon_forward := fun X Y Z => by
    apply StableAtlasFamilyHom.ext
    . funext i
      rcases i with (i | j) | k <;> rfl
    . intro i
      rcases i with (i | j) | k <;>
        simp [stableAtlasMergeHom, stableAtlasAssociator,
          stableAtlasAssociatorHom, stableAtlasBraiding,
          stableAtlasBraidingHom, stableAtlasMerge]
  hexagon_reverse := fun X Y Z => by
    apply StableAtlasFamilyHom.ext
    . funext i
      rcases i with i | (j | k) <;> rfl
    . intro i
      rcases i with i | (j | k) <;>
        simp [stableAtlasMergeHom, stableAtlasAssociator,
          stableAtlasAssociatorInv, stableAtlasBraiding,
          stableAtlasBraidingHom, stableAtlasMerge]

instance : SymmetricCategory StableAtlasFamily where
  symmetry X Y := by
    apply StableAtlasFamilyHom.ext
    . funext i
      cases i <;> rfl
    . intro i
      cases i <;>
        simp [stableAtlasBraided, stableAtlasBraiding,
          stableAtlasBraidingHom, stableAtlasMerge]

def IsAtlasMap : ObjectProperty Atl := fun A =>
  forall  v : extent A, Covered A A.P.folio.originElement v

abbrev AtlMap := IsAtlasMap.FullSubcategory

def AtlMapInc : AtlMap ==> Atl := ObjectProperty.iota IsAtlasMap

def IsAtlasMapTrav : ObjectProperty AtlTrav := fun A => IsAtlasMap A.obj
abbrev AtlTravMap := IsAtlasMapTrav.FullSubcategory

def AtlTravMapInc : AtlTravMap ==> AtlTrav := ObjectProperty.iota IsAtlasMapTrav

/-- The dominion of covered data in a cell. -/
def coveredDom (X : Atl) (x : X.E) : DomIns where
  toDom :=
    { Carrier := {t : X.G.obj x // Covered X x t}
      rank :=
        ({ toFun := Subtype.val
           inj' := Subtype.val_injective } :
          Function.Embedding {t : X.G.obj x // Covered X x t} (X.G.obj x)).trans
            (X.G.obj x).toDom.rank }

def coveredMap {X : Atl} {x y : X.E} (f : x --> y) :
    coveredDom X x --> coveredDom X y where
  toFun t := <|X.G.map f t.1, by
    rcases t.2 with <|k, l, h|>
    refine <|k, l, ?_|>
    have e : f >> X.P.folio.toOrigin y = X.P.folio.toOrigin x :=
      CategoryOfElements.ext _ _ _ (Subsingleton.elim _ _)
    change (X.G.map f >> X.G.map (X.P.folio.toOrigin y)) t.1 = _
    rw [<- X.G.map_comp, e]
    exact h|>
  inj' := fun a b h => Subtype.ext <| (X.G.map f).injective <| congrArg Subtype.val h

def coveredFunctor (X : Atl) : X.E ==> DomIns where
  obj := coveredDom X
  map := coveredMap
  map_id x := by
    apply DomIns.hom_ext
```

```
    intro t
    apply Subtype.ext
    change (X.G.map (Id x)) t.1 = t.1
    rw [X.G.map_id]
    change Function.Embedding.refl _ t.1 = t.1
    rfl
  map_comp f g := by
    apply DomIns.hom_ext
    intro t
    apply Subtype.ext
    change (X.G.map (f >> g)) t.1 =
      (X.G.map f >> X.G.map g) t.1
    rw [X.G.map_comp]

def chartAtlas (X : Atl) : Atl where
  P := X.P
  G := coveredFunctor X
  disjoint := by
    intro m i j hij x y h
    apply X.disjoint m i j hij x.1 y.1
    exact congrArg Subtype.val h

theorem chart_all_covered (X : Atl) (x : (chartAtlas X).E)
    (t : (chartAtlas X).G.obj x) : Covered (chartAtlas X) x t := by
  rcases t.2 with <|k, l, h|>
  let l' : territory (chartAtlas X) k := <|l, covered_region X k l|>
  exact <|k, l', Subtype.ext h|>

theorem chart_isAtlasMap (X : Atl) : IsAtlasMap (chartAtlas X) :=
  fun v => chart_all_covered X _ v

def chartCounit (X : Atl) : chartAtlas X --> X where
  P := Unit X.E
  A :=
    { app := fun _ =>
        { toFun := Subtype.val
          inj' := Subtype.val_injective }
      naturality := by intros; ext; rfl }

def chartMap {X Y : AtlTrav} (f : X --> Y) :
    chartAtlas X.obj --> chartAtlas Y.obj where
  P := f.1.P
  A :=
    { app := fun x =>
        { toFun := fun t => <|f.1.A.app x t.1, f.2.2.2 x t.1 t.2|>
          inj' := fun a b h => Subtype.ext <| (f.1.A.app x).injective <|
            congrArg Subtype.val h }
      naturality := by
        intro x y g
        apply DomIns.hom_ext
        intro t
        apply Subtype.ext
        exact congrFun (congrArg Function.Embedding.toFun (f.1.A.naturality g)) t.1 }

theorem chartMap_isTraversal {X Y : AtlTrav} (f : X --> Y) :
    IsTraversal (chartMap f) := by
  refine <|f.2.1, f.2.2.1, ?_|>
  intro x t _
  exact chart_all_covered Y.obj _ _

def Chr : AtlTrav ==> AtlTravMap where
  obj X := <|WideSubcategory.mk (chartAtlas X.obj), chart_isAtlasMap X.obj|>
  map f := <|chartMap f, chartMap_isTraversal f|>
  map_id _ := by
    apply Subtype.ext
    change chartMap (Id _) = AtlHom.identity _
    apply AtlHom.ext
    . rfl
    . apply heq_of_eq
      ext x t
      rfl
  map_comp f g := by
    apply Subtype.ext
```

```
    change chartMap (_ >> _) = AtlHom.comp (chartMap _) (chartMap _)
    apply AtlHom.ext
    . rfl
    . apply heq_of_eq
      ext x t
      rfl

/-- The formal construction realizes the proof sketch by retaining in each
cell exactly the data covered by final regions; its counit is the canonical
inclusion into the original atlas. -/

theorem originImage_origin (X : Atl) (v : extent X) :
    originImage X X.P.folio.originElement v = v := by
  have e : X.P.folio.toOrigin X.P.folio.originElement =
      Id X.P.folio.originElement :=
    CategoryOfElements.ext _ _ _ (Subsingleton.elim _ _)
  rw [originImage, storedOriginImage, e, X.G.map_id]
  change Function.Embedding.refl _ v = v
  rfl

theorem atlasMap_all_covered {X : Atl} (hX : IsAtlasMap X)
    (x : X.E) (t : X.G.obj x) : Covered X x t := by
  rcases hX (originImage X x t) with <|k, l, h|>
  exact <|k, l, by
    change originImage X X.P.folio.originElement (originImage X x t) =
      originImage X (X.P.cell X.P.folio.lastBase k) l at h
    simpa only [originImage_origin] using h|>

theorem chartCounit_isTraversal (X : Atl) : IsTraversal (chartCounit X) := by
  refine <|Function.injective_id, fun _ _ h => h, ?_|>
  intro x t _
  simpa [chartCounit] using t.2

def chartCounitTrav (X : AtlTrav) :
    (WideSubcategory.mk (chartAtlas X.obj) : AtlTrav) --> X :=
  <|chartCounit X.obj, chartCounit_isTraversal X.obj|>

def chartLift {A : AtlTravMap} {X : AtlTrav}
    (f : AtlTravMapInc.obj A --> X) :
    A --> Chr.obj X := by
  refine <|?_, ?_|>
  . exact
      { P := f.1.P
        A :=
          { app := fun x =>
              { toFun := fun t => <|f.1.A.app x t,
                  f.2.2.2 x t (atlasMap_all_covered A.property x t)|>
                inj' := by
                  intro a b h
                  apply (f.1.A.app x).injective
                  exact congrArg (fun z => z.1) h }
            naturality := by
              intro x y g
              apply DomIns.hom_ext
              intro t
              apply Subtype.ext
              exact congrFun (congrArg Function.Embedding.toFun (f.1.A.naturality g)) t } }
  . refine <|f.2.1, f.2.2.1, ?_|>
    intro x t _
    exact chart_all_covered X.obj _ _

def chartHomEquiv (A : AtlTravMap) (X : AtlTrav) :
    (AtlTravMapInc.obj A --> X) Equiv (A --> Chr.obj X) where
  toFun := chartLift
  invFun g := g >> chartCounitTrav X
  left_inv f := by
    apply Subtype.ext
    apply AtlHom.ext
    . rfl
    . apply heq_of_eq
      ext x t
      rfl
  right_inv g := by
```

```
    apply Subtype.ext
    apply AtlHom.ext
    . rfl
    . apply heq_of_eq
      ext x t
      apply Subtype.ext
      rfl

theorem chartHomEquiv_naturality_left {A A' : AtlTravMap} (f : A' --> A)
    {X : AtlTrav} (g : AtlTravMapInc.obj A --> X) :
    chartHomEquiv A' X (AtlTravMapInc.map f >> g) =
      f >> chartHomEquiv A X g := by
  apply Subtype.ext
  apply AtlHom.ext
  . rfl
  . apply heq_of_eq
    ext x t
    apply Subtype.ext
    rfl

theorem chartHomEquiv_naturality_right {A : AtlTravMap} {X X' : AtlTrav}
    (f : AtlTravMapInc.obj A --> X) (g : X --> X') :
    chartHomEquiv A X' (f >> g) = chartHomEquiv A X f >> Chr.map g := by
  apply Subtype.ext
  apply AtlHom.ext
  . rfl
  . apply heq_of_eq
    ext x t
    apply Subtype.ext
    rfl

def cartographyAdjunction : AtlTravMapInc -| Chr :=
  Adjunction.mkOfHomEquiv
    { homEquiv := chartHomEquiv
      homEquiv_naturality_left_symm := by
        intro X' X Y f g
        apply (chartHomEquiv X' Y).injective
        simpa using (chartHomEquiv_naturality_left f ((chartHomEquiv X Y).symm g)).symm
      homEquiv_naturality_right := by
        intro X Y Y' f g
        exact chartHomEquiv_naturality_right f g }

theorem cartographyLemma : Nonempty (AtlTravMapInc -| Chr) :=
  <|cartographyAdjunction|>

def extentMap {X Y : Atl} (f : X --> Y) : extent X --> extent Y :=
  f.A.app X.P.folio.originElement >>
    Y.G.map (Y.P.folio.toOrigin (f.P.obj X.P.folio.originElement))

theorem extentMap_originImage {X Y : Atl} (f : X --> Y) (x : X.E)
    (a : X.G.obj x) :
    extentMap f (originImage X x a) =
      originImage Y (f.P.obj x) (f.A.app x a) := by
  simp only [extentMap, originImage, storedOriginImage]
  have hn := f.A.naturality (X.P.folio.toOrigin x)
  have hc : f.P.map (X.P.folio.toOrigin x) >>
      Y.P.folio.toOrigin (f.P.obj X.P.folio.originElement) =
      Y.P.folio.toOrigin (f.P.obj x) := by
    apply CategoryOfElements.ext
    apply Subsingleton.elim
  rw [<- hc, Y.G.map_comp]
  exact congrFun (congrArg Function.Embedding.toFun
    (congrArg (fun k => k >>
      Y.G.map (Y.P.folio.toOrigin (f.P.obj X.P.folio.originElement))) hn)) a

theorem extentMap_id (X : Atl) : extentMap (Id X) = Id (extent X) := by
  apply DomIns.hom_ext
  intro t
  change (X.G.map (X.P.folio.toOrigin X.P.folio.originElement)) t = t
  simpa [originImage] using originImage_origin X t

theorem extentMap_comp {X Y Z : Atl} (f : X --> Y) (g : Y --> Z) :
    extentMap (f >> g) = extentMap f >> extentMap g := by
```

```
  apply DomIns.hom_ext
  intro t
  let oX := X.P.folio.originElement
  let y := f.P.obj oX
  let oY := Y.P.folio.originElement
  let z := g.P.obj y
  let z_0 := g.P.obj oY
  have hn := g.A.naturality (Y.P.folio.toOrigin y)
  have hz : g.P.map (Y.P.folio.toOrigin y) >> Z.P.folio.toOrigin z_0 =
      Z.P.folio.toOrigin z :=
    CategoryOfElements.ext _ _ _ (Subsingleton.elim _ _)
  have hn' : Y.G.map (Y.P.folio.toOrigin y) >> g.A.app oY =
      g.A.app y >> Z.G.map (g.P.map (Y.P.folio.toOrigin y)) := hn
  have hm : g.A.app y >> Z.G.map (Z.P.folio.toOrigin z) =
      Y.G.map (Y.P.folio.toOrigin y) >> g.A.app oY >>
        Z.G.map (Z.P.folio.toOrigin z_0) := by
    rw [<- Category.assoc, hn', Category.assoc, <- Z.G.map_comp, hz]
  exact congrFun (congrArg Function.Embedding.toFun
    (by simpa only [Category.assoc] using congrArg (fun q => f.A.app oX >> q) hm)) t

def Ex : Atl ==> DomIns where
  obj := extent
  map := extentMap
  map_id := extentMap_id
  map_comp := extentMap_comp

def stableCoalitionMap {X Y : StaAtlTrav} (f : X --> Y) :
    coveredDom X.obj.obj X.obj.obj.P.folio.originElement -->
      coveredDom Y.obj.obj Y.obj.obj.P.folio.originElement where
  toFun t := by
    have hc := f.1.2.2.2 X.obj.obj.P.folio.originElement t.1 t.2
    exact coveredMap (X := Y.obj.obj) (eqToHom f.2)
      <|f.1.1.A.app X.obj.obj.P.folio.originElement t.1, hc|>
  inj' := fun a b h => by
    apply Subtype.ext
    apply (f.1.1.A.app X.obj.obj.P.folio.originElement).injective
    apply (Y.obj.obj.G.map (eqToHom f.2)).injective
    exact congrArg Subtype.val h

theorem stableCoalitionMap_val {X Y : StaAtlTrav} (f : X --> Y)
    (t : coveredDom X.obj.obj X.obj.obj.P.folio.originElement) :
    (stableCoalitionMap f t).1 = extentMap f.1.1 t.1 := by
  change Y.obj.obj.G.map (eqToHom f.2)
      (f.1.1.A.app X.obj.obj.P.folio.originElement t.1) =
    Y.obj.obj.G.map (Y.obj.obj.P.folio.toOrigin
      (f.1.1.P.obj X.obj.obj.P.folio.originElement))
      (f.1.1.A.app X.obj.obj.P.folio.originElement t.1)
  have hmor : eqToHom f.2 = Y.obj.obj.P.folio.toOrigin
      (f.1.1.P.obj X.obj.obj.P.folio.originElement) := by
    apply CategoryOfElements.ext
    apply Subsingleton.elim
  rw [hmor]

/-- `Coa` is written directly on the covered origins.  This is definitionally
the object part of `Ex  o  Chr`; stability makes its action on arrows reduce to
the component at the origin. -/
def Coa : StaAtlTrav ==> DomIns where
  obj X := coveredDom X.obj.obj X.obj.obj.P.folio.originElement
  map := stableCoalitionMap
  map_id X := by
    apply DomIns.hom_ext
    intro t
    apply Subtype.ext
    rw [stableCoalitionMap_val]
    exact congrFun (congrArg Function.Embedding.toFun (extentMap_id X.obj.obj)) t.1
  map_comp f g := by
    apply DomIns.hom_ext
    intro t
    apply Subtype.ext
    rw [stableCoalitionMap_val]
    change extentMap (f.1.1 >> g.1.1) t.1 =
      (stableCoalitionMap g (stableCoalitionMap f t)).1
    rw [stableCoalitionMap_val, stableCoalitionMap_val]
```

```
    exact congrFun (congrArg Function.Embedding.toFun
      (extentMap_comp f.1.1 g.1.1)) t.1

def coalition (X : Atl) : DomIns := (Coa.obj
  (WideSubcategory.mk (WideSubcategory.mk X) : StaAtlTrav))

/-- The initial dominion. -/
def emptyDominion : DomIns where
  toDom :=
    { Carrier := Empty
      rank :=
        { toFun := fun x => nomatch x
          inj' := fun x => nomatch x } }

def singletonChain : Chain where
  Obj := Unit
  linearOrder := inferInstance
  wellFoundedLT := inferInstance
  orderType_lt := by
    have hpow : Ordinal.omega0 ^ (1 : Ordinal) <
        Ordinal.omega0 ^ Ordinal.omega0 :=
      (Ordinal.opow_lt_opow_iff_right Ordinal.one_lt_omega0).2
        Ordinal.one_lt_omega0
    have homega : Ordinal.omega0 < Ordinal.omega0 ^ Ordinal.omega0 := by
      simpa only [Ordinal.opow_one] using hpow
    simpa using lt_trans Ordinal.one_lt_omega0 homega

def singletonObjEquiv : singletonChain.Obj Equiv Unit := Equiv.refl _

def onePageFolio : Folio where
  length := 1
  positive := by omega
  core := (Functor.const (Fin 1)).obj (op singletonChain)
  originEquiv := singletonObjEquiv

def onePagePag : Pag := <|onePageFolio|>

theorem onePage_cell_unique (x : onePagePag.E) :
    x = onePageFolio.originElement := by
  let hbase : x.1 = onePageFolio.originBase := by
    apply unop_injective
    apply Fin.eq_zero
  apply Functor.Elements.ext x onePageFolio.originElement hbase
  exact onePageFolio.origin_unique _

def dominionAtlas (X : DomIns) : Atl where
  P := onePagePag
  G := (Functor.const onePagePag.E).obj X
  disjoint := by
    intro m i j hij
    exfalso
    apply hij
    apply onePageFolio.originEquiv.injective
    exact Subsingleton.elim _ _

theorem dominionAtlas_isMap (X : DomIns) : IsAtlasMap (dominionAtlas X) := by
  intro v
  let k : TerritoryIndex (dominionAtlas X) :=
    onePageFolio.originValue
  refine <|k, v, ?_|>
  change originImage (dominionAtlas X)
    (dominionAtlas X).P.folio.originElement v =
      originImage (dominionAtlas X)
        ((dominionAtlas X).P.cell (dominionAtlas X).P.folio.lastBase k) v
  rw [originImage_origin]
  have hc : (dominionAtlas X).P.cell (dominionAtlas X).P.folio.lastBase k =
      (dominionAtlas X).P.folio.originElement := onePage_cell_unique _
  rw [hc, originImage_origin]

def dominionMap {X Y : DomIns} (f : X --> Y) :
    dominionAtlas X --> dominionAtlas Y where
  P := Unit onePagePag.E
  A :=
```

```
    { app := fun _ => f
      naturality := by intros; apply DomIns.hom_ext; intro; rfl }

theorem dominionMap_isTraversal {X Y : DomIns} (f : X --> Y) :
    IsTraversal (dominionMap f) := by
  refine <|Function.injective_id, fun _ _ h => h, ?_|>
  intro x t _
  exact atlasMap_all_covered (dominionAtlas_isMap Y) _ _

theorem dominionMap_isStable {X Y : DomIns} (f : X --> Y) :
    IsStableTraversal
      (X := WideSubcategory.mk (dominionAtlas X))
      (Y := WideSubcategory.mk (dominionAtlas Y))
      <|dominionMap f, dominionMap_isTraversal f|> := rfl

def DomInc : DomIns ==> StaAtlTrav where
  obj X := WideSubcategory.mk (WideSubcategory.mk (dominionAtlas X))
  map f := <|<|dominionMap f, dominionMap_isTraversal f|>,
    dominionMap_isStable f|>
  map_id _ := by
    apply Subtype.ext
    apply Subtype.ext
    apply AtlHom.ext
    . rfl
    . apply heq_of_eq
      ext
      rfl
  map_comp f g := by
    apply Subtype.ext
    apply Subtype.ext
    apply AtlHom.ext
    . rfl
    . apply heq_of_eq
      ext x t
      change (f >> g) t = (f >> g) t
      rfl

def onePageToAtlasP (Y : Atl) : onePagePag.E ==> Y.E :=
  (Functor.const onePagePag.E).obj Y.P.folio.originElement

theorem onePageToAtlasP_injective (Y : Atl) :
    Function.Injective (onePageToAtlasP Y).obj := by
  intro x y _
  exact onePage_cell_unique x |>.trans (onePage_cell_unique y).symm

theorem onePage_elementLT_false (X : DomIns) (x y : (dominionAtlas X).E) :
    not  elementLT (dominionAtlas X) x y := by
  intro h
  have hxy : x = y := onePage_cell_unique x |>.trans (onePage_cell_unique y).symm
  subst y
  let m := (dominionAtlas X).P.folio.commonBase x.1 x.1
  letI : LinearOrder ((dominionAtlas X).H.obj m) :=
    ((dominionAtlas X).P.folio.core.obj m.unop).unop.linearOrder
  exact lt_irrefl _ h

/-- A stable traversal from a one-page atlas determines an embedding into
the covered part of the target extent. -/
def domIncToCoa {X : DomIns} {Y : StaAtlTrav} (f : DomInc.obj X --> Y) :
    X --> Coa.obj Y where
  toFun x := by
    have hc := f.1.2.2.2 onePageFolio.originElement x
      (atlasMap_all_covered (dominionAtlas_isMap X) _ x)
    exact coveredMap (X := Y.obj.obj) (eqToHom f.2)
      <|f.1.1.A.app onePageFolio.originElement x, hc|>
  inj' := fun a b h => by
    apply (f.1.1.A.app onePageFolio.originElement).injective
    apply (Y.obj.obj.G.map (eqToHom f.2)).injective
    exact congrArg Subtype.val h

/-- Conversely, an embedding into the coalition supplies the unique stable
traversal from the corresponding one-page atlas. -/
def coaToDomInc {X : DomIns} {Y : StaAtlTrav} (f : X --> Coa.obj Y) :
    DomInc.obj X --> Y := by
```

```
  let p : (dominionAtlas X).E ==> Y.obj.obj.P.E := onePageToAtlasP Y.obj.obj
  let a : (dominionAtlas X).G --> p then Y.obj.obj.G :=
    { app := fun _ =>
        { toFun := fun x => (f x).1
          inj' := fun x y h => f.injective (Subtype.ext h) }
      naturality := by
        intro x y g
        apply DomIns.hom_ext
        intro t
        have hxy : x = y := onePage_cell_unique x |>.trans
          (onePage_cell_unique y).symm
        subst y
        have hg : g = Id x := by
          apply CategoryOfElements.ext
          apply Subsingleton.elim
        subst g
        simp [p] }
  let h : dominionAtlas X --> Y.obj.obj := <|p, a|>
  have htrav : IsTraversal h := by
    refine <|onePageToAtlasP_injective Y.obj.obj, ?_, ?_|>
    . intro x y hxy
      exact (onePage_elementLT_false X x y hxy).elim
    . intro x t _
      change Covered Y.obj.obj Y.obj.obj.P.folio.originElement (f t).1
      exact (f t).2
  exact <|<|h, htrav|>, rfl|>

def coaDomIncIso (X : DomIns) : Coa.obj (DomInc.obj X) Iso X where
  hom :=
    { toFun := fun x => x.1
      inj' := fun x y h => Subtype.ext h }
  inv :=
    { toFun := fun x => <|x, atlasMap_all_covered
        (dominionAtlas_isMap X) onePageFolio.originElement x|>
      inj' := fun x y h => congrArg Subtype.val h }
  hom_inv_id := by
    apply DomIns.hom_ext
    intro x
    apply Subtype.ext
    rfl
  inv_hom_id := by
    apply DomIns.hom_ext
    intro x
    rfl

/-- The hom-set equivalence expressing that one-page insertion is left
adjoint to coalization. -/
def domIncCoaHomEquiv (X : DomIns) (Y : StaAtlTrav) :
    (DomInc.obj X --> Y) Equiv (X --> Coa.obj Y) where
  toFun := domIncToCoa
  invFun := coaToDomInc
  left_inv := by
    intro f
    apply Subtype.ext
    apply Subtype.ext
    have hP : (coaToDomInc (domIncToCoa f)).1.1.P = f.1.1.P := by
      exact CategoryTheory.Functor.ext
        (fun x => f.2.symm.trans
          (congrArg f.1.1.P.obj (onePage_cell_unique x).symm))
        (fun _ _ _ => by
          apply CategoryOfElements.ext
          apply Subsingleton.elim)
    apply AtlHom.ext _ _ hP
    apply NatTrans.hext_right _ _
      (congrArg (fun Q => Q then Y.obj.obj.G) hP)
    intro x
    dsimp [coaToDomInc, domIncToCoa, onePageToAtlasP]
    have hx : x = onePageFolio.originElement := onePage_cell_unique x
    subst x
    let e := f.1.1.A.app onePageFolio.originElement
    have hrec :
        { toFun := fun x =>
            (coveredMap (X := Y.obj.obj) (eqToHom f.2)
```

```
              <|e x, by
                exact f.1.2.2.2 onePageFolio.originElement x
                  (atlasMap_all_covered (dominionAtlas_isMap X) _ x)|>).1
          inj' := by
            intro a b hab
            apply e.injective
            apply (Y.obj.obj.G.map (eqToHom f.2)).injective
            exact hab } = e >> Y.obj.obj.G.map (eqToHom f.2) := by
      apply DomIns.hom_ext
      intro t
      rfl
    exact (heq_of_eq hrec).trans
      (embedding_comp_map_eqToHom_heq Y.obj.obj.G f.2 e)
  right_inv := by
    intro f
    apply DomIns.hom_ext
    intro x
    apply Subtype.ext
    change Y.obj.obj.G.map (Id Y.obj.obj.P.folio.originElement) (f x).1 = (f x).1
    exact congrFun (congrArg Function.Embedding.toFun
      (Y.obj.obj.G.map_id Y.obj.obj.P.folio.originElement)) (f x).1

theorem domIncCoaHomEquiv_naturality_left {X X' : DomIns} (f : X' --> X)
    {Y : StaAtlTrav} (g : DomInc.obj X --> Y) :
    domIncCoaHomEquiv X' Y (DomInc.map f >> g) =
      f >> domIncCoaHomEquiv X Y g := by
  apply DomIns.hom_ext
  intro x
  apply Subtype.ext
  rfl

theorem domIncCoaHomEquiv_naturality_right {X : DomIns} {Y Y' : StaAtlTrav}
    (f : DomInc.obj X --> Y) (g : Y --> Y') :
    domIncCoaHomEquiv X Y' (f >> g) =
      domIncCoaHomEquiv X Y f >> Coa.map g := by
  apply DomIns.hom_ext
  intro x
  apply Subtype.ext
  let t : Coa.obj (DomInc.obj X) :=
    <|x, atlasMap_all_covered
      (dominionAtlas_isMap X) onePageFolio.originElement x|>
  change (Coa.map (f >> g) t).1 = (Coa.map g (Coa.map f t)).1
  exact congrArg Subtype.val <| congrFun
    (congrArg Function.Embedding.toFun (Coa.map_comp f g)) t

/-- The unconditional adjunction promised by the Domanial Inclusion
construction. -/
def dominionAdjunction : DomInc -| Coa :=
  Adjunction.mkOfHomEquiv
    { homEquiv := domIncCoaHomEquiv
      homEquiv_naturality_left_symm := by
        intro X' X Y f g
        apply (domIncCoaHomEquiv X' Y).injective
        simpa using
          (domIncCoaHomEquiv_naturality_left f
            ((domIncCoaHomEquiv X Y).symm g)).symm
      homEquiv_naturality_right := by
        intro X Y Y' f g
        exact domIncCoaHomEquiv_naturality_right f g }

theorem dominionLemma : Nonempty (DomInc -| Coa) :=
  <|dominionAdjunction|>

abbrev DaTra := Atlop ==> Type

def Yo : Atl ==> DaTra := yoneda

/-- A concrete certificate of the statement that `DaTra` is the displayed
presheaf category. -/
def datraToposPresentation : DaTra EquivCat (Atlop ==> Type) :=
  CategoryTheory.Equivalence.refl

structure Navigation (D : DaTra) where
```

```
  A : Atl
  hom : Yo.obj A --> D
  mono : Mono hom

attribute [instance] Navigation.mono

structure Expedition (D : DaTra) extends Navigation D where
  atlasMap : IsAtlasMap A

abbrev DaTrap := AtlTrapop ==> Type
abbrev DaTrav := AtlTravop ==> Type

/-- Presheaves on the wide category of atlas traversals. -/
abbrev AtlTravPSh := AtlTravop ==> Type

/-- Forget the action of an atlas presheaf on non-traversal arrows. -/
def DaTra.restrictToAtlTrav : DaTra ==> AtlTravPSh :=
  (Functor.whiskeringLeft AtlTravop Atlop Type).obj AtlTravInc.op

/-- The promised identification is definitional after restricting the
indexing category. -/
def DaTrav.atlTravPShEquiv : DaTrav EquivCat AtlTravPSh :=
  CategoryTheory.Equivalence.refl

/-- Presheaves on atlas federations of stable atlas traversals. -/
abbrev StaDaTravPresheaf := StableAtlasFamilyop ==> Type 3

/-- The all-objects wrapper gives Day convolution its own monoidal structure,
separate from the pointwise cartesian structure on a raw functor category. -/
def IsStableDataTransversal : ObjectProperty StaDaTravPresheaf := fun _ => True

/-- Stable data transversals: presheaves whose indexing arrows are stable atlas
traversals.  Stability is therefore enforced by the source category. -/
abbrev StaDaTrav := IsStableDataTransversal.FullSubcategory

abbrev StaDaTravInc : StaDaTrav ==> StaDaTravPresheaf :=
  IsStableDataTransversal.iota

def IsDaTraMap : ObjectProperty DaTra := fun D =>
  forall  nav : Navigation D, IsAtlasMap nav.A

abbrev DaTraMap := IsDaTraMap.FullSubcategory

def AtlI : Atl := dominionAtlas emptyDominion

theorem AtlI_eq_domInc_zero :
    AtlI = (DomInc.obj emptyDominion).obj.obj := rfl

def Folio.pageValue (W : Folio) (m : Fin W.length) :
    (W.core.obj m).unop.Obj := by
  letI : NeZero W.length := <|Nat.ne_of_gt W.positive|>
  let q : (W.core.obj m).unop --> (W.core.obj W.originIndex).unop :=
    (W.core.map (homOfLE (Fin.zero_le m))).unop
  exact Classical.choose (q.point_surjective W.originValue)

theorem Folio.map_unop_comp_apply (W : Folio) {i j k : Fin W.length}
    (hik : i --> k) (hij : i --> j) (hjk : j --> k)
    (z : (W.core.obj k).unop.Obj) :
    (W.core.map hik).unop z =
      (W.core.map hij).unop ((W.core.map hjk).unop z) := by
  have e : hik = hij >> hjk := Subsingleton.elim _ _
  subst hik
  have h := congrArg Quiver.Hom.unop (W.core.map_comp hij hjk)
  exact congrFun (congrArg ConHom.toFun h) z

def bouquetLength (X Y : Folio) : Nat := max X.length Y.length + 1

def bouquetDepth (X Y : Folio) : Nat := max X.length Y.length

theorem bouquetLength_pos (X Y : Folio) : 0 < bouquetLength X Y := by
  simp [bouquetLength]

instance (X Y : Folio) : NeZero (bouquetLength X Y) :=
```

```
  <|Nat.ne_of_gt (bouquetLength_pos X Y)|>

def bouquetChain (X Y : Folio) : Fin (bouquetLength X Y) -> Chain :=
  Fin.cases singletonChain (fun m : Fin (bouquetDepth X Y) =>
    Chain.sum (X.core.obj (X.paddedIndex m.1)).unop
      (Y.core.obj (Y.paddedIndex m.1)).unop)

def bouquetValue (X Y : Folio) (m : Fin (bouquetLength X Y)) :
    (bouquetChain X Y m).Obj :=
  Fin.cases () (fun n => toLex (Sum.inl (X.pageValue (X.paddedIndex n.1)))) m

def bouquetConMap (X Y : Folio) {i j : Fin (bouquetLength X Y)} (h : i <= j) :
    bouquetChain X Y j --> bouquetChain X Y i := by
  revert j
  refine Fin.cases ?_ (fun i => ?_) i
  . rename_i j
    intro h
    refine
      { toFun := fun _ => ()
        monotone := fun _ _ _ => le_rfl
        point_surjective := fun z => by
          refine <|bouquetValue X Y j, ?_|>
          change (() : Unit) = z
          exact Subsingleton.elim _ _ }
  . rename_i j
    intro h
    revert h
    refine Fin.cases (by
      intro h
      change i.1 + 1 <= 0 at h
      omega) (fun j => ?_) j
    intro h
    have hij : i <= j := Fin.succ_le_succ_iff.mp h
    let hx : X.paddedIndex i.1 <= X.paddedIndex j.1 := X.paddedIndex_mono hij
    let hy : Y.paddedIndex i.1 <= Y.paddedIndex j.1 := Y.paddedIndex_mono hij
    exact ConHom.sum (X.core.map (homOfLE hx)).unop
      (Y.core.map (homOfLE hy)).unop

def bouquetFolio (X Y : Folio) : Folio where
  length := bouquetLength X Y
  positive := bouquetLength_pos X Y
  core :=
    { obj := fun m => op (bouquetChain X Y m)
      map := fun f => (bouquetConMap X Y (leOfHom f)).op
      map_id := by
        intro m
        refine Fin.cases ?_ (fun m => ?_) m
        . apply Quiver.Hom.unop_inj
          apply ConHom.ext
          intro z
          change (() : Unit) = ()
          rfl
        apply Quiver.Hom.unop_inj
        apply ConHom.ext
        intro w
        generalize hz : ofLex w = z
        rcases z with z | z
        . have hz' : w = toLex (Sum.inl z) := ofLex.injective (by simpa using hz)
          subst w
          simp [bouquetConMap, ConHom.sum]
          change toLex (Sum.inl z) = toLex (Sum.inl z)
          rfl
        . have hz' : w = toLex (Sum.inr z) := ofLex.injective (by simpa using hz)
          subst w
          simp [bouquetConMap, ConHom.sum]
          change toLex (Sum.inr z) = toLex (Sum.inr z)
          rfl
      map_comp := by
        intro i j k f g
        letI : NeZero (bouquetLength X Y) := <|Nat.ne_of_gt (bouquetLength_pos X Y)|>
        revert j k
        refine Fin.cases ?_ (fun i => ?_) i
        . rename_i jTarget kTarget
```

```
          intro f g
          apply Quiver.Hom.unop_inj
          apply ConHom.ext
          intro z
          change (() : Unit) = ()
          rfl
        . rename_i iOrig jTarget kTarget
          intro f g
          have hj : jTarget != 0 := by
            intro e
            have hf := leOfHom f
            rw [e] at hf
            change i.1 + 1 <= 0 at hf
            omega
          obtain <|j, rfl|> := Fin.eq_succ_of_ne_zero hj
          have hk : kTarget != 0 := by
            intro e
            have hg := leOfHom g
            rw [e] at hg
            change j.1 + 1 <= 0 at hg
            omega
          obtain <|k, rfl|> := Fin.eq_succ_of_ne_zero hk
          apply Quiver.Hom.unop_inj
          apply ConHom.ext
          intro w
          generalize hz : ofLex w = z
          rcases z with z | z
          . have hz' : w = toLex (Sum.inl z) := ofLex.injective (by simpa using hz)
            subst w
            simpa only [bouquetConMap, ConHom.sum] using congrArg
              (fun q => toLex (Sum.inl q)) (X.map_unop_comp_apply _ _ _ z)
          . have hz' : w = toLex (Sum.inr z) := ofLex.injective (by simpa using hz)
            subst w
            simpa only [bouquetConMap, ConHom.sum] using congrArg
              (fun q => toLex (Sum.inr q)) (Y.map_unop_comp_apply _ _ _ z) }
  originEquiv := by
    exact singletonObjEquiv

def domSum (X Y : DomIns) : DomIns where
  toDom :=
    { Carrier := Sum X Y
      rank :=
        { toFun := fun z => match z with
            | .inl x => 2 * X.toDom.rank x
            | .inr y => 2 * Y.toDom.rank y + 1
          inj' := by
            intro a b h
            rcases a with a | a <;> rcases b with b | b
            . simp only at h
              exact congrArg Sum.inl (X.toDom.rank.injective (by omega))
            . simp only at h
              omega
            . simp only at h
              omega
            . simp only at h
              exact congrArg Sum.inr (Y.toDom.rank.injective (by omega)) } }

def domSum.inl (X Y : DomIns) : X --> domSum X Y where
  toFun := Sum.inl
  inj' := Sum.inl_injective

def domSum.inr (X Y : DomIns) : Y --> domSum X Y where
  toFun := Sum.inr
  inj' := Sum.inr_injective

def domSum.map {X_1 X_2 Y_1 Y_2 : DomIns} (f : X_1 --> Y_1) (g : X_2 --> Y_2) :
    domSum X_1 X_2 --> domSum Y_1 Y_2 where
  toFun := Sum.map f g
  inj' := by
    intro a b h
    rcases a with a | a <;> rcases b with b | b
    . exact congrArg Sum.inl (f.injective (Sum.inl.inj h))
    . exact Sum.noConfusion h
```

```
    . exact Sum.noConfusion h
    . exact congrArg Sum.inr (g.injective (Sum.inr.inj h))

def tallMergePage (X Y : TallAtlas) : Nat -> Type
  | 0 => Unit
  | n + 1 => Sum (X.H.obj (op n)) (Y.H.obj (op n))

def tallMergePageMap (X Y : TallAtlas) {i j : Nat} (h : j <= i) :
    tallMergePage X Y i -> tallMergePage X Y j := by
  induction j with
  | zero => exact fun _ => ()
  | succ j =>
      induction i with
      | zero => omega
      | succ i =>
          exact Sum.map
            (X.H.map (homOfLE (Nat.succ_le_succ_iff.mp h)).op)
            (Y.H.map (homOfLE (Nat.succ_le_succ_iff.mp h)).op)

def tallMergeH (X Y : TallAtlas) : Natop ==> Type where
  obj n := tallMergePage X Y n.unop
  map f := tallMergePageMap X Y (leOfHom f.unop)
  map_id n := by
    funext z
    rcases n with <|n|>
    cases n with
    | zero => rfl
    | succ n =>
        rcases z with z | z
        . apply congrArg Sum.inl
          simp
        . apply congrArg Sum.inr
          simp
  map_comp := by
    intro ni nj nk f g
    funext z
    rcases ni with <|i|>
    rcases nj with <|j|>
    rcases nk with <|k|>
    cases k with
    | zero => rfl
    | succ k =>
        have hj : j != 0 := by
          intro e
          subst j
          have hg := leOfHom g.unop
          change k + 1 <= 0 at hg
          omega
        obtain <|j, rfl|> := Nat.exists_eq_succ_of_ne_zero hj
        have hi : i != 0 := by
          intro e
          subst i
          have hf := leOfHom f.unop
          change j + 1 <= 0 at hf
          omega
        obtain <|i, rfl|> := Nat.exists_eq_succ_of_ne_zero hi
        let fij : (op i : Natop) --> op j :=
          (homOfLE (Nat.succ_le_succ_iff.mp (leOfHom f.unop))).op
        let fjk : (op j : Natop) --> op k :=
          (homOfLE (Nat.succ_le_succ_iff.mp (leOfHom g.unop))).op
        let fik : (op i : Natop) --> op k :=
          (homOfLE (Nat.succ_le_succ_iff.mp (leOfHom (f >> g).unop))).op
        have hcomp : fik = fij >> fjk := Subsingleton.elim _ _
        rcases z with z | z
        . apply congrArg Sum.inl
          change X.H.map fik z = X.H.map fjk (X.H.map fij z)
          rw [hcomp, X.H.map_comp]
          rfl
        . apply congrArg Sum.inr
          change Y.H.map fik z = Y.H.map fjk (Y.H.map fij z)
          rw [hcomp, Y.H.map_comp]
          rfl

```

```
def bouquetPag (X Y : Atl) : Pag := <|bouquetFolio X.P.folio Y.P.folio|>

def bouquetLeftIndex (X Y : Atl) (m : Fin X.P.folio.length) :
    Fin (bouquetLength X.P.folio Y.P.folio) :=
  <|m.1 + 1, by simp [bouquetLength]|>

def bouquetRightIndex (X Y : Atl) (m : Fin Y.P.folio.length) :
    Fin (bouquetLength X.P.folio Y.P.folio) :=
  <|m.1 + 1, by simp [bouquetLength]|>

def bouquetLeftPred (X Y : Atl) (m : Fin X.P.folio.length) :
    Fin (bouquetDepth X.P.folio Y.P.folio) :=
  <|m.1, lt_of_lt_of_le m.2 (le_max_left _ _)|>

def bouquetRightPred (X Y : Atl) (m : Fin Y.P.folio.length) :
    Fin (bouquetDepth X.P.folio Y.P.folio) :=
  <|m.1, lt_of_lt_of_le m.2 (le_max_right _ _)|>

@[simp]
theorem bouquetLeftIndex_eq_succ (X Y : Atl) (m : Fin X.P.folio.length) :
    bouquetLeftIndex X Y m = (bouquetLeftPred X Y m).succ := rfl

@[simp]
theorem bouquetRightIndex_eq_succ (X Y : Atl) (m : Fin Y.P.folio.length) :
    bouquetRightIndex X Y m = (bouquetRightPred X Y m).succ := rfl

def bouquetLeftElement (X Y : Atl) (x : X.E) : (bouquetPag X Y).E := by
  refine <|op (bouquetLeftPred X Y x.1.unop).succ, ?_|>
  change (bouquetChain X.P.folio Y.P.folio
    (bouquetLeftPred X Y x.1.unop).succ).Obj
  change Lex (Sum
    ((X.P.folio.core.obj (X.P.folio.paddedIndex x.1.unop.1)).unop.Obj)
    ((Y.P.folio.core.obj (Y.P.folio.paddedIndex x.1.unop.1)).unop.Obj))
  exact toLex (Sum.inl (X.P.H.map
    (eqToHom (congrArg op (X.P.folio.paddedIndex_fin x.1.unop).symm)) x.2))

def bouquetRightElement (X Y : Atl) (y : Y.E) : (bouquetPag X Y).E := by
  refine <|op (bouquetRightPred X Y y.1.unop).succ, ?_|>
  change (bouquetChain X.P.folio Y.P.folio
    (bouquetRightPred X Y y.1.unop).succ).Obj
  change Lex (Sum
    ((X.P.folio.core.obj (X.P.folio.paddedIndex y.1.unop.1)).unop.Obj)
    ((Y.P.folio.core.obj (Y.P.folio.paddedIndex y.1.unop.1)).unop.Obj))
  exact toLex (Sum.inr (Y.P.H.map
    (eqToHom (congrArg op (Y.P.folio.paddedIndex_fin y.1.unop).symm)) y.2))

def bouquetLeftMapHom (X Y : Atl) {x y : X.E} (q : x --> y) :
    bouquetLeftElement X Y x --> bouquetLeftElement X Y y := by
  rcases x with <|<|mx|>, kx|>
  rcases y with <|<|my|>, ky|>
  let hxy : my <= mx := leOfHom (Quiver.Hom.unop q.val)
  let h : bouquetLeftPred X Y my <= bouquetLeftPred X Y mx := by
    change my.1 <= mx.1
    exact hxy
  refine CategoryOfElements.homMk _ _
    (homOfLE (Fin.succ_le_succ_iff.mpr h)).op ?_
  dsimp only [bouquetLeftElement]
  simp only [bouquetLeftPred, bouquetPag, bouquetFolio, Folio.H,
    bouquetChain, bouquetConMap, Fin.cases_succ, ConHom.sum, Functor.leftOp_map,
    Functor.comp_obj, Functor.comp_map, Quiver.Hom.unop_op, Tra]
  change toLex (Sum.inl _) = toLex (Sum.inl _)
  congr 2
  have hqv : X.P.H.map q.val kx = ky := q.property
  rw [<- hqv]
  let hp : X.P.folio.paddedIndex my.1 <= X.P.folio.paddedIndex mx.1 :=
    X.P.folio.paddedIndex_mono hxy
  change X.P.H.map (homOfLE hp).op
    (X.P.H.map (eqToHom _) kx) = X.P.H.map (eqToHom _)
      (X.P.H.map q.val kx)
  rw [<- FunctorToTypes.map_comp_apply]
  rw [<- FunctorToTypes.map_comp_apply]
  congr 1

```

```
def bouquetRightMapHom (X Y : Atl) {x y : Y.E} (q : x --> y) :
    bouquetRightElement X Y x --> bouquetRightElement X Y y := by
  rcases x with <|<|mx|>, kx|>
  rcases y with <|<|my|>, ky|>
  let hxy : my <= mx := leOfHom (Quiver.Hom.unop q.val)
  let h : bouquetRightPred X Y my <= bouquetRightPred X Y mx := by
    change my.1 <= mx.1
    exact hxy
  refine CategoryOfElements.homMk _ _
    (homOfLE (Fin.succ_le_succ_iff.mpr h)).op ?_
  dsimp only [bouquetRightElement]
  simp only [bouquetRightPred, bouquetPag, bouquetFolio, Folio.H,
    bouquetChain, bouquetConMap, Fin.cases_succ, ConHom.sum, Functor.leftOp_map,
    Functor.comp_obj, Functor.comp_map, Quiver.Hom.unop_op, Tra]
  change toLex (Sum.inr _) = toLex (Sum.inr _)
  congr 2
  have hqv : Y.P.H.map q.val kx = ky := q.property
  rw [<- hqv]
  let hp : Y.P.folio.paddedIndex my.1 <= Y.P.folio.paddedIndex mx.1 :=
    Y.P.folio.paddedIndex_mono hxy
  change Y.P.H.map (homOfLE hp).op
    (Y.P.H.map (eqToHom _) kx) = Y.P.H.map (eqToHom _)
      (Y.P.H.map q.val kx)
  rw [<- FunctorToTypes.map_comp_apply]
  rw [<- FunctorToTypes.map_comp_apply]
  congr 1

def bouquetLeftElements (X Y : Atl) : X.E ==> (bouquetPag X Y).E where
  obj := bouquetLeftElement X Y
  map := bouquetLeftMapHom X Y
  map_id _ := by
    apply CategoryOfElements.ext
    apply Subsingleton.elim
  map_comp _ _ := by
    apply CategoryOfElements.ext
    apply Subsingleton.elim

def bouquetRightElements (X Y : Atl) : Y.E ==> (bouquetPag X Y).E where
  obj := bouquetRightElement X Y
  map := bouquetRightMapHom X Y
  map_id _ := by
    apply CategoryOfElements.ext
    apply Subsingleton.elim
  map_comp _ _ := by
    apply CategoryOfElements.ext
    apply Subsingleton.elim

def imageDom {A R : DomIns} (f : A --> R) : DomIns where
  toDom :=
    { Carrier := Set.range f
      rank :=
        ({ toFun := Subtype.val
           inj' := Subtype.val_injective } : Set.range f >-> R).trans R.toDom.rank }

def imageDom.inclusion {A R : DomIns} (f : A --> R) : imageDom f --> R where
  toFun := Subtype.val
  inj' := Subtype.val_injective

def imageDom.map {A B R : DomIns} (f : A --> R) (g : B --> R)
    (h : Set.range f subset Set.range g) : imageDom f --> imageDom g where
  toFun z := <|z.1, h z.2|>
  inj' := by
    intro a b e
    apply Subtype.ext
    exact congrArg (fun q => q.1) e

def imageDom.mapAcross {A B R S : DomIns} (f : A --> R) (g : B --> S)
    (h : R --> S) (factor : forall  a, exists  b, h (f a) = g b) :
    imageDom f --> imageDom g where
  toFun z := <|h z.1, by
    rcases z.2 with <|a, ha|>
    rcases factor a with <|b, hb|>
    exact <|b, (congrArg h ha.symm |>.trans hb).symm|>|>
```

```
  inj' := by
    intro a b e
    apply Subtype.ext
    apply h.injective
    exact congrArg Subtype.val e

def tallMergeRawDom (X Y : TallAtlas) (x : (tallMergeH X Y).Elements) : DomIns := by
  rcases x with <|<|n|>, v|>
  cases n with
  | zero => exact domSum X.extent Y.extent
  | succ n =>
      exact match v with
        | .inl k => X.G.obj <|op n, k|>
        | .inr k => Y.G.obj <|op n, k|>

def tallMergeRootMap (X Y : TallAtlas) (x : (tallMergeH X Y).Elements) :
    tallMergeRawDom X Y x --> domSum X.extent Y.extent := by
  rcases x with <|<|n|>, v|>
  cases n with
  | zero => exact Id _
  | succ n =>
      exact match v with
        | .inl k => X.G.map (X.toOrigin <|op n, k|>) >>
            domSum.inl X.extent Y.extent
        | .inr k => Y.G.map (Y.toOrigin <|op n, k|>) >>
            domSum.inr X.extent Y.extent

def tallMergeDom (X Y : TallAtlas) (x : (tallMergeH X Y).Elements) : DomIns :=
  imageDom (tallMergeRootMap X Y x)

theorem tallMergeRange_mono (X Y : TallAtlas)
    {x y : (tallMergeH X Y).Elements} (f : x --> y) :
    Set.range (tallMergeRootMap X Y x) subset
      Set.range (tallMergeRootMap X Y y) := by
  intro z hz
  rcases x with <|<|mx|>, vx|>
  rcases y with <|<|my|>, vy|>
  rcases hz with <|a, rfl|>
  cases mx with
  | zero =>
      have hmy : my = 0 := by
        have hf := leOfHom f.val.unop
        change my <= 0 at hf
        omega
      subst my
      exact <|a, rfl|>
  | succ n =>
      cases my with
      | zero => exact <|tallMergeRootMap X Y <|op n.succ, vx|> a, rfl|>
      | succ p =>
          have hpn : p <= n := Nat.succ_le_succ_iff.mp (leOfHom f.val.unop)
          rcases vx with k | k
          . have hfv : Sum.inl (X.H.map (homOfLE hpn).op k) = vy := by
              simpa [tallMergeH, tallMergePageMap] using f.property
            subst vy
            let q := CategoryOfElements.homMk
              (<|op n, k|> : X.E)
              (<|op p, X.H.map (homOfLE hpn).op k|> : X.E)
              (homOfLE hpn).op rfl
            refine <|X.G.map q a, ?_|>
            change Sum.inl (X.G.map (X.toOrigin _) (X.G.map q a)) =
              Sum.inl (X.G.map (X.toOrigin _) a)
            apply congrArg Sum.inl
            have hc : q >> X.toOrigin _ = X.toOrigin _ :=
              CategoryOfElements.ext X.H _ _ (Subsingleton.elim _ _)
            have hm := X.G.map_comp q (X.toOrigin _)
            rw [hc] at hm
            exact congrFun (congrArg Function.Embedding.toFun hm.symm) a
          . have hfv : Sum.inr (Y.H.map (homOfLE hpn).op k) = vy := by
              simpa [tallMergeH, tallMergePageMap] using f.property
            subst vy
            let q := CategoryOfElements.homMk
              (<|op n, k|> : Y.E)
```

```
              (<|op p, Y.H.map (homOfLE hpn).op k|> : Y.E)
              (homOfLE hpn).op rfl
            refine <|Y.G.map q a, ?_|>
            change Sum.inr (Y.G.map (Y.toOrigin _) (Y.G.map q a)) =
              Sum.inr (Y.G.map (Y.toOrigin _) a)
            apply congrArg Sum.inr
            have hc : q >> Y.toOrigin _ = Y.toOrigin _ :=
              CategoryOfElements.ext Y.H _ _ (Subsingleton.elim _ _)
            have hm := Y.G.map_comp q (Y.toOrigin _)
            rw [hc] at hm
            exact congrFun (congrArg Function.Embedding.toFun hm.symm) a

def tallMergeImageMap (X Y : TallAtlas)
    {x y : (tallMergeH X Y).Elements} (f : x --> y) :
    tallMergeDom X Y x --> tallMergeDom X Y y :=
  imageDom.map _ _ (tallMergeRange_mono X Y f)

def tallMergeG (X Y : TallAtlas) : (tallMergeH X Y).Elements ==> DomIns where
  obj := tallMergeDom X Y
  map := tallMergeImageMap X Y
  map_id _ := by
    apply DomIns.hom_ext
    intro z
    apply Subtype.ext
    rfl
  map_comp _ _ := by
    apply DomIns.hom_ext
    intro z
    apply Subtype.ext
    rfl

def tallMergeCellLT (X Y : TallAtlas)
    (x y : (tallMergeH X Y).Elements) : Prop := by
  rcases x with <|<|nx|>, vx|>
  rcases y with <|<|ny|>, vy|>
  cases nx with
  | zero => exact False
  | succ nx =>
      cases ny with
      | zero => exact False
      | succ ny =>
          exact match vx, vy with
            | .inl k, .inl l => X.cellLT <|op nx, k|> <|op ny, l|>
            | .inl _, .inr _ => True
            | .inr _, .inl _ => False
            | .inr k, .inr l => Y.cellLT <|op nx, k|> <|op ny, l|>

def tallMerge (X Y : TallAtlas) : TallAtlas where
  H := tallMergeH X Y
  originValue := ()
  originUnique x := by
    change Unit at x
    cases x
    rfl
  G := tallMergeG X Y
  cellLT := tallMergeCellLT X Y
  coveredExtent t := match t.1 with
    | .inl a => X.coveredExtent a
    | .inr b => Y.coveredExtent b

def tallMergeLeftObj (X Y : TallAtlas) (x : X.E) : (tallMerge X Y).E :=
  <|op (x.1.unop + 1), Sum.inl x.2|>

def tallMergeRightObj (X Y : TallAtlas) (y : Y.E) : (tallMerge X Y).E :=
  <|op (y.1.unop + 1), Sum.inr y.2|>

def tallMergeLeftMap (X Y : TallAtlas) {x y : X.E} (f : x --> y) :
    tallMergeLeftObj X Y x --> tallMergeLeftObj X Y y := by
  rcases x with <|<|n|>, kx|>
  rcases y with <|<|m|>, ky|>
  have hmn : m <= n := leOfHom f.val.unop
  refine CategoryOfElements.homMk _ _
    (homOfLE (Nat.succ_le_succ hmn)).op ?_
```

```
  change Sum.inl (X.H.map (homOfLE hmn).op kx) = Sum.inl ky
  congr 1
  have ef : f.val = (homOfLE hmn).op := Subsingleton.elim _ _
  rw [<- ef]
  exact f.property

def tallMergeRightMap (X Y : TallAtlas) {x y : Y.E} (f : x --> y) :
    tallMergeRightObj X Y x --> tallMergeRightObj X Y y := by
  rcases x with <|<|n|>, kx|>
  rcases y with <|<|m|>, ky|>
  have hmn : m <= n := leOfHom f.val.unop
  refine CategoryOfElements.homMk _ _
    (homOfLE (Nat.succ_le_succ hmn)).op ?_
  change Sum.inr (Y.H.map (homOfLE hmn).op kx) = Sum.inr ky
  congr 1
  have ef : f.val = (homOfLE hmn).op := Subsingleton.elim _ _
  rw [<- ef]
  exact f.property

def tallMergeLeft (X Y : TallAtlas) : X.E ==> (tallMerge X Y).E where
  obj := tallMergeLeftObj X Y
  map := tallMergeLeftMap X Y
  map_id _ := by apply CategoryOfElements.ext; apply Subsingleton.elim
  map_comp _ _ := by apply CategoryOfElements.ext; apply Subsingleton.elim

def tallMergeRight (X Y : TallAtlas) : Y.E ==> (tallMerge X Y).E where
  obj := tallMergeRightObj X Y
  map := tallMergeRightMap X Y
  map_id _ := by apply CategoryOfElements.ext; apply Subsingleton.elim
  map_comp _ _ := by apply CategoryOfElements.ext; apply Subsingleton.elim

def tallHorPObj {X_1 X_2 Y_1 Y_2 : TallAtlas} (f : X_1 --> Y_1) (g : X_2 --> Y_2)
    (x : (tallMerge X_1 X_2).E) : (tallMerge Y_1 Y_2).E := by
  rcases x with <|<|n|>, v|>
  cases n with
  | zero => exact (tallMerge Y_1 Y_2).originElement
  | succ n =>
      exact match v with
        | .inl k => tallMergeLeftObj Y_1 Y_2 (f.P.obj <|op n, k|>)
        | .inr k => tallMergeRightObj Y_1 Y_2 (g.P.obj <|op n, k|>)

def tallHorPHom {X_1 X_2 Y_1 Y_2 : TallAtlas} (f : X_1 --> Y_1) (g : X_2 --> Y_2)
    {x y : (tallMerge X_1 X_2).E} (q : x --> y) :
    tallHorPObj f g x --> tallHorPObj f g y := by
  rcases x with <|<|mx|>, vx|>
  rcases y with <|<|my|>, vy|>
  cases mx with
  | zero =>
      have hmy : my = 0 := by
        have hq := leOfHom q.val.unop
        change my <= 0 at hq
        omega
      subst my
      exact Id _
  | succ n =>
      cases my with
      | zero => exact (tallMerge Y_1 Y_2).toOrigin _
      | succ p =>
          have hpn : p <= n := Nat.succ_le_succ_iff.mp (leOfHom q.val.unop)
          rcases vx with k | k
          . have hqv : Sum.inl (X_1.H.map (homOfLE hpn).op k) = vy := by
              simpa [tallMerge, tallMergeH, tallMergePageMap] using q.property
            subst vy
            let r := CategoryOfElements.homMk
              (<|op n, k|> : X_1.E)
              (<|op p, X_1.H.map (homOfLE hpn).op k|> : X_1.E)
              (homOfLE hpn).op rfl
            exact tallMergeLeftMap Y_1 Y_2 (f.P.map r)
          . have hqv : Sum.inr (X_2.H.map (homOfLE hpn).op k) = vy := by
              simpa [tallMerge, tallMergeH, tallMergePageMap] using q.property
            subst vy
            let r := CategoryOfElements.homMk
              (<|op n, k|> : X_2.E)
```

```
              (<|op p, X_2.H.map (homOfLE hpn).op k|> : X_2.E)
              (homOfLE hpn).op rfl
            exact tallMergeRightMap Y_1 Y_2 (g.P.map r)

def tallHorP {X_1 X_2 Y_1 Y_2 : TallAtlas} (f : X_1 --> Y_1) (g : X_2 --> Y_2) :
    (tallMerge X_1 X_2).E ==> (tallMerge Y_1 Y_2).E where
  obj := tallHorPObj f g
  map := tallHorPHom f g
  map_id _ := by apply CategoryOfElements.ext; apply Subsingleton.elim
  map_comp _ _ := by apply CategoryOfElements.ext; apply Subsingleton.elim

def TallAtlas.originImage (X : TallAtlas) (x : X.E) (a : X.G.obj x) : X.extent :=
  X.G.map (X.toOrigin x) a

@[simp]
theorem TallAtlas.originImage_origin (X : TallAtlas) (a : X.extent) :
    X.originImage X.originElement a = a := by
  change X.G.map (X.toOrigin X.originElement) a = a
  have h : X.toOrigin X.originElement = Id X.originElement :=
    Subsingleton.elim _ _
  rw [h, X.G.map_id]
  rfl

def TallAtlas.extentMap {X Y : TallAtlas} (f : X --> Y) : X.extent --> Y.extent :=
  f.A.app X.originElement >> Y.G.map (Y.toOrigin (f.P.obj X.originElement))

@[simp]
theorem TallAtlas.extentMap_id (X : TallAtlas) :
    TallAtlas.extentMap (Id X) = Id X.extent := by
  apply DomIns.hom_ext
  intro a
  change X.G.map (X.toOrigin X.originElement) a = a
  have h : X.toOrigin X.originElement = Id X.originElement := Subsingleton.elim _ _
  rw [h]
  rw [X.G.map_id]
  rfl

theorem TallAtlas.extentMap_originImage {X Y : TallAtlas} (f : X --> Y)
    (x : X.E) (a : X.G.obj x) :
    TallAtlas.extentMap f (X.originImage x a) =
      Y.originImage (f.P.obj x) (f.A.app x a) := by
  simp only [TallAtlas.extentMap, TallAtlas.originImage]
  have hn := f.A.naturality (X.toOrigin x)
  have hc : f.P.map (X.toOrigin x) >> Y.toOrigin (f.P.obj X.originElement) =
      Y.toOrigin (f.P.obj x) := by
    apply CategoryOfElements.ext
    apply Subsingleton.elim
  rw [<- hc, Y.G.map_comp]
  exact congrFun (congrArg Function.Embedding.toFun
    (congrArg (fun k => k >> Y.G.map (Y.toOrigin (f.P.obj X.originElement))) hn)) a

@[simp]
theorem TallAtlas.extentMap_comp {X Y Z : TallAtlas} (f : X --> Y) (g : Y --> Z) :
    TallAtlas.extentMap (f >> g) =
      TallAtlas.extentMap f >> TallAtlas.extentMap g := by
  simp only [TallAtlas.extentMap]
  have hn := g.A.naturality (Y.toOrigin (f.P.obj X.originElement))
  simp only [Functor.comp_map] at hn
  have hc : g.P.map (Y.toOrigin (f.P.obj X.originElement)) >>
      Z.toOrigin (g.P.obj Y.originElement) =
      Z.toOrigin (g.P.obj (f.P.obj X.originElement)) := by
    apply CategoryOfElements.ext
    apply Subsingleton.elim
  change f.A.app X.originElement >> g.A.app (f.P.obj X.originElement) >>
      Z.G.map (Z.toOrigin (g.P.obj (f.P.obj X.originElement))) =
    (f.A.app X.originElement >> Y.G.map (Y.toOrigin (f.P.obj X.originElement))) >>
      g.A.app Y.originElement >> Z.G.map (Z.toOrigin (g.P.obj Y.originElement))
  simp only [Category.assoc]
  rw [<- Category.assoc (Y.G.map (Y.toOrigin (f.P.obj X.originElement)))
    (g.A.app Y.originElement) (Z.G.map (Z.toOrigin (g.P.obj Y.originElement)))]
  rw [hn]
  rw [Category.assoc (g.A.app (f.P.obj X.originElement))]
  rw [<- Z.G.map_comp]
```

```
  rw [hc]

def tallHorExtentMap {X_1 X_2 Y_1 Y_2 : TallAtlas}
    (f : X_1 --> Y_1) (g : X_2 --> Y_2) :
    domSum X_1.extent X_2.extent --> domSum Y_1.extent Y_2.extent :=
  domSum.map (TallAtlas.extentMap f) (TallAtlas.extentMap g)

theorem tallHorRootFactor {X_1 X_2 Y_1 Y_2 : TallAtlas}
    (f : X_1 --> Y_1) (g : X_2 --> Y_2)
    (x : (tallMerge X_1 X_2).E) (a : tallMergeRawDom X_1 X_2 x) :
    exists  b : tallMergeRawDom Y_1 Y_2 (tallHorPObj f g x),
      tallHorExtentMap f g (tallMergeRootMap X_1 X_2 x a) =
        tallMergeRootMap Y_1 Y_2 (tallHorPObj f g x) b := by
  rcases x with <|<|n|>, v|>
  cases n with
  | zero => exact <|tallHorExtentMap f g a, rfl|>
  | succ n =>
      rcases v with k | k
      . let x_0 : X_1.E := <|op n, k|>
        let y_0 := f.P.obj x_0
        refine <|f.A.app x_0 a, ?_|>
        change Sum.inl (TallAtlas.extentMap f (X_1.originImage x_0 a)) =
          Sum.inl (Y_1.originImage y_0 (f.A.app x_0 a))
        exact congrArg Sum.inl (TallAtlas.extentMap_originImage f x_0 a)
      . let x_0 : X_2.E := <|op n, k|>
        let y_0 := g.P.obj x_0
        refine <|g.A.app x_0 a, ?_|>
        change Sum.inr (TallAtlas.extentMap g (X_2.originImage x_0 a)) =
          Sum.inr (Y_2.originImage y_0 (g.A.app x_0 a))
        exact congrArg Sum.inr (TallAtlas.extentMap_originImage g x_0 a)

def tallHorA {X_1 X_2 Y_1 Y_2 : TallAtlas} (f : X_1 --> Y_1) (g : X_2 --> Y_2) :
    (tallMerge X_1 X_2).G --> tallHorP f g then (tallMerge Y_1 Y_2).G where
  app x := imageDom.mapAcross
    (tallMergeRootMap X_1 X_2 x)
    (tallMergeRootMap Y_1 Y_2 (tallHorPObj f g x))
    (tallHorExtentMap f g) (tallHorRootFactor f g x)
  naturality := by
    intro x y q
    apply DomIns.hom_ext
    intro z
    apply Subtype.ext
    rfl

def tallHorMap {X_1 X_2 Y_1 Y_2 : TallAtlas} (f : X_1 --> Y_1) (g : X_2 --> Y_2) :
    tallMerge X_1 X_2 --> tallMerge Y_1 Y_2 where
  P := tallHorP f g
  A := tallHorA f g

@[simp]
theorem tallHorMap_extentMap_val {X_1 X_2 Y_1 Y_2 : TallAtlas}
    (f : X_1 --> Y_1) (g : X_2 --> Y_2)
    (z : (tallMerge X_1 X_2).extent) :
    (TallAtlas.extentMap (tallHorMap f g) z).1 =
      tallHorExtentMap f g z.1 := by
  rfl

theorem tallHorMap_id (X Y : TallAtlas) :
    tallHorMap (Id X) (Id Y) = Id (tallMerge X Y) := by
  have hP : tallHorP (Id X) (Id Y) = Unit (tallMerge X Y).E := by
    exact CategoryTheory.Functor.ext (F := tallHorP (Id X) (Id Y))
      (G := Unit (tallMerge X Y).E) (fun x => by
      rcases x with <|<|n|>, v|>
      cases n with
      | zero => rfl
      | succ n => cases v <;> rfl)
  have hA : HEq (tallHorMap (Id X) (Id Y)).A
      (Id (tallMerge X Y) : TallAtlasHom _ _).A := by
    apply NatTrans.hext_right _ _
      (congrArg (fun P => P then (tallMerge X Y).G) hP)
    intro x
    rcases x with <|<|n|>, v|>
    cases n with
```

```
    | zero =>
        apply heq_of_eq
        apply DomIns.hom_ext
        intro z
        apply Subtype.ext
        simp only [tallHorMap, tallHorA, imageDom.mapAcross, tallHorExtentMap]
        rw [TallAtlas.extentMap_id, TallAtlas.extentMap_id]
        change Sum.map (Id X.extent) (Id Y.extent) z.1 = z.1
        rcases z.1 with a | b <;> rfl
    | succ n =>
        rcases v with v | v <;>
          apply heq_of_eq <;>
          apply DomIns.hom_ext <;>
          intro z <;>
          apply Subtype.ext <;>
          simp only [tallHorMap, tallHorA, imageDom.mapAcross, tallHorExtentMap] <;>
          rw [TallAtlas.extentMap_id, TallAtlas.extentMap_id] <;>
          change Sum.map (Id X.extent) (Id Y.extent) z.1 = z.1 <;>
          rcases z.1 with a | b <;> rfl
  exact TallAtlasHom.ext _ _ hP hA

theorem tallHorMap_comp {X_1 X_2 Y_1 Y_2 Z_1 Z_2 : TallAtlas}
    (f_1 : X_1 --> Y_1) (f_2 : Y_1 --> Z_1) (g_1 : X_2 --> Y_2) (g_2 : Y_2 --> Z_2) :
    tallHorMap (f_1 >> f_2) (g_1 >> g_2) =
      tallHorMap f_1 g_1 >> tallHorMap f_2 g_2 := by
  have hP : tallHorP (f_1 >> f_2) (g_1 >> g_2) =
      tallHorP f_1 g_1 then tallHorP f_2 g_2 := by
    exact CategoryTheory.Functor.ext
      (F := tallHorP (f_1 >> f_2) (g_1 >> g_2))
      (G := tallHorP f_1 g_1 then tallHorP f_2 g_2) (fun x => by
        rcases x with <|<|n|>, v|>
        cases n with
        | zero => rfl
        | succ n => cases v <;> rfl)
  have hA : HEq (tallHorMap (f_1 >> f_2) (g_1 >> g_2)).A
      (tallHorMap f_1 g_1 >> tallHorMap f_2 g_2).A := by
    apply NatTrans.hext_right _ _
      (congrArg (fun P => P then (tallMerge Z_1 Z_2).G) hP)
    intro x
    rcases x with <|<|n|>, v|>
    cases n with
    | zero =>
        apply heq_of_eq
        apply DomIns.hom_ext
        intro z
        apply Subtype.ext
        simp only [tallHorMap, tallHorA, imageDom.mapAcross, tallHorExtentMap,
          TallAtlas.extentMap_comp]
        change (domSum.map
            (TallAtlas.extentMap f_1 >> TallAtlas.extentMap f_2)
            (TallAtlas.extentMap g_1 >> TallAtlas.extentMap g_2)) z.1 =
          domSum.map (TallAtlas.extentMap f_2) (TallAtlas.extentMap g_2)
            (domSum.map (TallAtlas.extentMap f_1) (TallAtlas.extentMap g_1) z.1)
        rcases z.1 with a | b <;> rfl
    | succ n =>
        rcases v with v | v <;>
          apply heq_of_eq <;>
          apply DomIns.hom_ext <;>
          intro z <;>
          apply Subtype.ext <;>
          simp only [tallHorMap, tallHorA, imageDom.mapAcross, tallHorExtentMap,
            TallAtlas.extentMap_comp] <;>
          change (domSum.map
              (TallAtlas.extentMap f_1 >> TallAtlas.extentMap f_2)
              (TallAtlas.extentMap g_1 >> TallAtlas.extentMap g_2)) z.1 =
            domSum.map (TallAtlas.extentMap f_2) (TallAtlas.extentMap g_2)
              (domSum.map (TallAtlas.extentMap f_1) (TallAtlas.extentMap g_1) z.1) <;>
          rcases z.1 with a | b <;> rfl
  exact TallAtlasHom.ext _ _ hP hA

/-- The horizontal sum on raw infinite-spine presentations.  This is the
implementation bifunctor from which the coherent atlas operation is derived. -/
def TallAtlHorSum : TallAtlas * TallAtlas ==> TallAtlas where
```

```
  obj X := tallMerge X.1 X.2
  map f := tallHorMap f.1 f.2
  map_id X := tallHorMap_id X.1 X.2
  map_comp f g := tallHorMap_comp f.1 g.1 f.2 g.2

def domSum.swap (X Y : DomIns) : domSum X Y --> domSum Y X where
  toFun z := z.swap
  inj' := by
    intro a b h
    rcases a with a | a <;> rcases b with b | b
    . exact congrArg Sum.inl (Sum.inr.inj h)
    . cases h
    . cases h
    . exact congrArg Sum.inr (Sum.inl.inj h)

def tallSwapPObj (X Y : TallAtlas) (x : (tallMerge X Y).E) :
    (tallMerge Y X).E := by
  rcases x with <|<|n|>, v|>
  cases n with
  | zero => exact (tallMerge Y X).originElement
  | succ n =>
      exact match v with
        | .inl k => tallMergeRightObj Y X <|op n, k|>
        | .inr k => tallMergeLeftObj Y X <|op n, k|>

def tallSwapPHom (X Y : TallAtlas) {x y : (tallMerge X Y).E} (q : x --> y) :
    tallSwapPObj X Y x --> tallSwapPObj X Y y := by
  rcases x with <|<|mx|>, vx|>
  rcases y with <|<|my|>, vy|>
  cases mx with
  | zero =>
      have hmy : my = 0 := by
        have hq := leOfHom q.val.unop
        change my <= 0 at hq
        omega
      subst my
      exact Id _
  | succ n =>
      cases my with
      | zero => exact (tallMerge Y X).toOrigin _
      | succ p =>
          have hpn : p <= n := Nat.succ_le_succ_iff.mp (leOfHom q.val.unop)
          rcases vx with k | k
          . have hqv : Sum.inl (X.H.map (homOfLE hpn).op k) = vy := by
              simpa [tallMerge, tallMergeH, tallMergePageMap] using q.property
            subst vy
            exact tallMergeRightMap Y X (CategoryOfElements.homMk _ _
              (homOfLE hpn).op rfl)
          . have hqv : Sum.inr (Y.H.map (homOfLE hpn).op k) = vy := by
              simpa [tallMerge, tallMergeH, tallMergePageMap] using q.property
            subst vy
            exact tallMergeLeftMap Y X (CategoryOfElements.homMk _ _
              (homOfLE hpn).op rfl)

def tallSwapP (X Y : TallAtlas) : (tallMerge X Y).E ==> (tallMerge Y X).E where
  obj := tallSwapPObj X Y
  map := tallSwapPHom X Y
  map_id _ := by apply CategoryOfElements.ext; apply Subsingleton.elim
  map_comp _ _ := by apply CategoryOfElements.ext; apply Subsingleton.elim

theorem tallSwapRootFactor (X Y : TallAtlas) (x : (tallMerge X Y).E)
    (a : tallMergeRawDom X Y x) :
    exists  b : tallMergeRawDom Y X (tallSwapPObj X Y x),
      domSum.swap X.extent Y.extent (tallMergeRootMap X Y x a) =
        tallMergeRootMap Y X (tallSwapPObj X Y x) b := by
  rcases x with <|<|n|>, v|>
  cases n with
  | zero => exact <|a.swap, rfl|>
  | succ n =>
      rcases v with k | k
      . exact <|a, rfl|>
      . exact <|a, rfl|>

```

```
def TallAtlBrd (X Y : TallAtlas) : tallMerge X Y --> tallMerge Y X where
  P := tallSwapP X Y
  A :=
    { app := fun x => imageDom.mapAcross
        (tallMergeRootMap X Y x)
        (tallMergeRootMap Y X (tallSwapPObj X Y x))
        (domSum.swap X.extent Y.extent) (tallSwapRootFactor X Y x)
      naturality := by
        intro x y q
        apply DomIns.hom_ext
        intro z
        apply Subtype.ext
        rfl }

theorem TallAtlBrd_involutive (X Y : TallAtlas) :
    TallAtlBrd X Y >> TallAtlBrd Y X = Id (tallMerge X Y) := by
  have hP : tallSwapP X Y then tallSwapP Y X = Unit (tallMerge X Y).E := by
    exact CategoryTheory.Functor.ext (fun x => by
      rcases x with <|<|n|>, v|>
      cases n with
      | zero => rfl
      | succ n => cases v <;> rfl)
  apply TallAtlasHom.ext _ _ hP
  apply NatTrans.hext_right _ _
    (congrArg (fun P => P then (tallMerge X Y).G) hP)
  intro x
  rcases x with <|<|n|>, v|>
  cases n with
  | zero =>
      apply heq_of_eq
      apply DomIns.hom_ext
      intro z
      apply Subtype.ext
      simp only [TallAtlBrd, imageDom.mapAcross]
      change domSum.swap Y.extent X.extent
          (domSum.swap X.extent Y.extent z.1) = z.1
      rcases z.1 with a | b <;> rfl
  | succ n =>
      rcases v with v | v <;>
        apply heq_of_eq <;>
        apply DomIns.hom_ext <;>
        intro z <;>
        apply Subtype.ext <;>
        simp only [TallAtlBrd, imageDom.mapAcross] <;>
        change domSum.swap Y.extent X.extent
            (domSum.swap X.extent Y.extent z.1) = z.1 <;>
        rcases z.1 with a | b <;> rfl

def TallAtlBrdIso (X Y : TallAtlas) : tallMerge X Y Iso tallMerge Y X where
  hom := TallAtlBrd X Y
  inv := TallAtlBrd Y X
  hom_inv_id := TallAtlBrd_involutive X Y
  inv_hom_id := TallAtlBrd_involutive Y X

/-- The empty atlas regarded on its full spine. -/
def TallAtlasI : TallAtlas := AtlI.tall

theorem domIns_eqToHom_apply {A B : DomIns} (h : A = B) (a : A) :
    (eqToHom h : A --> B) a = h transport a := by
  cases h
  rfl

theorem castDep_symm_cast {A : Sort u} (P : A -> Sort*) {x y : A}
    (h : x = y) (a : P y) : h transport (h.symm transport a) = a := by
  cases h
  rfl

def bouquetRawDomIndex (X Y : Atl) :
    (m : Fin (bouquetLength X.P.folio Y.P.folio)) ->
      (bouquetFolio X.P.folio Y.P.folio).H.obj (op m) -> DomIns :=
  Fin.cases (fun _ => domSum (extent X) (extent Y)) (fun n v =>
    match ofLex v with
    | .inl k => X.G.obj (X.P.cell (op (X.P.folio.paddedIndex n.1)) k)
```

```
    | .inr k => Y.G.obj (Y.P.cell (op (Y.P.folio.paddedIndex n.1)) k))

theorem bouquetLeftRawDom_eq (X Y : Atl) (x : X.E) :
    bouquetRawDomIndex X Y (bouquetLeftElement X Y x).1.unop
      (bouquetLeftElement X Y x).2 = X.G.obj x := by
  rcases x with <|<|m|>, k|>
  simp only [bouquetLeftElement, bouquetLeftPred, bouquetRawDomIndex,
    bouquetChain, Fin.cases_succ]
  split
  . rename_i k' hk
    change Sum.inl _ = Sum.inl k' at hk
    have hk' := Sum.inl.inj hk
    subst k'
    congr 1
    apply Functor.Elements.ext (F := X.P.H) _ _
      (congrArg op (X.P.folio.paddedIndex_fin m))
    dsimp only [Pag.cell]
    change X.P.H.map (eqToHom _) (X.P.H.map (eqToHom _) k) = k
    rw [<- FunctorToTypes.map_comp_apply]
    simp
  . rename_i k' hk
    change Sum.inl _ = Sum.inr k' at hk
    cases hk

theorem bouquetRightRawDom_eq (X Y : Atl) (y : Y.E) :
    bouquetRawDomIndex X Y (bouquetRightElement X Y y).1.unop
      (bouquetRightElement X Y y).2 = Y.G.obj y := by
  rcases y with <|<|m|>, k|>
  simp only [bouquetRightElement, bouquetRightPred, bouquetRawDomIndex,
    bouquetChain, Fin.cases_succ]
  split
  . rename_i k' hk
    change Sum.inr _ = Sum.inl k' at hk
    cases hk
  . rename_i k' hk
    change Sum.inr _ = Sum.inr k' at hk
    have hk' := Sum.inr.inj hk
    subst k'
    congr 1
    apply Functor.Elements.ext (F := Y.P.H) _ _
      (congrArg op (Y.P.folio.paddedIndex_fin m))
    dsimp only [Pag.cell]
    change Y.P.H.map (eqToHom _) (Y.P.H.map (eqToHom _) k) = k
    rw [<- FunctorToTypes.map_comp_apply]
    simp

def paddedElement (X : Atl) (x : X.E) : X.E :=
  X.P.cell (op (X.P.folio.paddedIndex x.1.unop.1))
    (X.P.H.map
      (eqToHom (congrArg op (X.P.folio.paddedIndex_fin x.1.unop).symm)) x.2)

theorem paddedElement_eq (X : Atl) (x : X.E) : paddedElement X x = x := by
  apply Functor.Elements.ext (F := X.P.H) _ _
    (congrArg op (X.P.folio.paddedIndex_fin x.1.unop))
  dsimp only [paddedElement, Pag.cell]
  change X.P.H.map (eqToHom _) (X.P.H.map (eqToHom _) x.2) = x.2
  rw [<- FunctorToTypes.map_comp_apply]
  simp

def bouquetRootIndex (X Y : Atl) :
    forall  (m : Fin (bouquetLength X.P.folio Y.P.folio))
      (v : (bouquetFolio X.P.folio Y.P.folio).H.obj (op m)),
      bouquetRawDomIndex X Y m v --> domSum (extent X) (extent Y) := by
  intro m
  induction m using Fin.cases with
  | zero => intro v; exact Id _
  | succ n =>
    intro v
    change (match ofLex v with
      | .inl k => X.G.obj (X.P.cell (op (X.P.folio.paddedIndex n.1)) k)
      | .inr k => Y.G.obj (Y.P.cell (op (Y.P.folio.paddedIndex n.1)) k))
      --> domSum (extent X) (extent Y)
    exact match ofLex v with
```

```
      | .inl k => X.G.map (X.P.cellToOrigin
          (op (X.P.folio.paddedIndex n.1)) k) >>
          domSum.inl (extent X) (extent Y)
      | .inr k => Y.G.map (Y.P.cellToOrigin
          (op (Y.P.folio.paddedIndex n.1)) k) >>
          domSum.inr (extent X) (extent Y)

@[simp]
theorem bouquetRootIndex_succ_left (X Y : Atl)
    (n : Fin (bouquetDepth X.P.folio Y.P.folio))
    (k : X.P.H.obj (op (X.P.folio.paddedIndex n.1)))
    (a : X.G.obj (X.P.cell (op (X.P.folio.paddedIndex n.1)) k)) :
    bouquetRootIndex X Y n.succ (toLex (Sum.inl k)) a =
      Sum.inl (X.G.map
        (X.P.cellToOrigin (op (X.P.folio.paddedIndex n.1)) k) a) := by
  rfl

@[simp]
theorem bouquetRootIndex_succ_right (X Y : Atl)
    (n : Fin (bouquetDepth X.P.folio Y.P.folio))
    (k : Y.P.H.obj (op (Y.P.folio.paddedIndex n.1)))
    (a : Y.G.obj (Y.P.cell (op (Y.P.folio.paddedIndex n.1)) k)) :
    bouquetRootIndex X Y n.succ (toLex (Sum.inr k)) a =
      Sum.inr (Y.G.map
        (Y.P.cellToOrigin (op (Y.P.folio.paddedIndex n.1)) k) a) := by
  rfl

theorem bouquetRootIndex_left (X Y : Atl) (x : X.E)
    (a : bouquetRawDomIndex X Y (bouquetLeftElement X Y x).1.unop
      (bouquetLeftElement X Y x).2) :
    bouquetRootIndex X Y (bouquetLeftElement X Y x).1.unop
        (bouquetLeftElement X Y x).2 a =
      Sum.inl (originImage X x (bouquetLeftRawDom_eq X Y x transport a)) := by
  rcases x with <|<|m|>, k|>
  change bouquetRootIndex X Y (bouquetLeftPred X Y m).succ
      (toLex (Sum.inl (X.P.H.map
        (eqToHom (congrArg op (X.P.folio.paddedIndex_fin m).symm)) k))) a = _
  rw [bouquetRootIndex_succ_left]
  congr 1
  change X.G.map (X.P.folio.toOrigin (paddedElement X <|op m, k|>)) a =
    X.G.map (X.P.folio.toOrigin <|op m, k|>)
      (bouquetLeftRawDom_eq X Y <|op m, k|> transport a)
  let q := paddedElement_eq X (<|op m, k|> : X.E)
  have hto : eqToHom q >> X.P.folio.toOrigin <|op m, k|> =
      X.P.folio.toOrigin (paddedElement X <|op m, k|>) := by
    apply CategoryOfElements.ext
    apply Subsingleton.elim
  rw [<- hto, X.G.map_comp]
  change X.G.map (X.P.folio.toOrigin <|op m, k|>)
      (X.G.map (eqToHom q) a) =
    X.G.map (X.P.folio.toOrigin <|op m, k|>)
      (bouquetLeftRawDom_eq X Y <|op m, k|> transport a)
  apply congrArg (fun z => X.G.map (X.P.folio.toOrigin <|op m, k|>) z)
  have hmap := eqToHom_map X.G q
  rw [hmap]
  have heq : congrArg X.G.obj q = bouquetLeftRawDom_eq X Y <|op m, k|> :=
    Subsingleton.elim _ _
  cases heq
  exact domIns_eqToHom_apply _ _

theorem bouquetRootIndex_right (X Y : Atl) (y : Y.E)
    (a : bouquetRawDomIndex X Y (bouquetRightElement X Y y).1.unop
      (bouquetRightElement X Y y).2) :
    bouquetRootIndex X Y (bouquetRightElement X Y y).1.unop
        (bouquetRightElement X Y y).2 a =
      Sum.inr (originImage Y y (bouquetRightRawDom_eq X Y y transport a)) := by
  rcases y with <|<|m|>, k|>
  change bouquetRootIndex X Y (bouquetRightPred X Y m).succ
      (toLex (Sum.inr (Y.P.H.map
        (eqToHom (congrArg op (Y.P.folio.paddedIndex_fin m).symm)) k))) a = _
  rw [bouquetRootIndex_succ_right]
  congr 1
  change Y.G.map (Y.P.folio.toOrigin (paddedElement Y <|op m, k|>)) a =
```

```
    Y.G.map (Y.P.folio.toOrigin <|op m, k|>)
      (bouquetRightRawDom_eq X Y <|op m, k|> transport a)
  let q := paddedElement_eq Y (<|op m, k|> : Y.E)
  have hto : eqToHom q >> Y.P.folio.toOrigin <|op m, k|> =
      Y.P.folio.toOrigin (paddedElement Y <|op m, k|>) := by
    apply CategoryOfElements.ext
    apply Subsingleton.elim
  rw [<- hto, Y.G.map_comp]
  change Y.G.map (Y.P.folio.toOrigin <|op m, k|>)
      (Y.G.map (eqToHom q) a) =
    Y.G.map (Y.P.folio.toOrigin <|op m, k|>)
      (bouquetRightRawDom_eq X Y <|op m, k|> transport a)
  apply congrArg (fun z => Y.G.map (Y.P.folio.toOrigin <|op m, k|>) z)
  have hmap := eqToHom_map Y.G q
  rw [hmap]
  have heq : congrArg Y.G.obj q = bouquetRightRawDom_eq X Y <|op m, k|> :=
    Subsingleton.elim _ _
  cases heq
  exact domIns_eqToHom_apply _ _

def bouquetDom (X Y : Atl) (x : (bouquetPag X Y).E) : DomIns :=
  imageDom (bouquetRootIndex X Y x.1.unop x.2)

theorem bouquetRange_mono (X Y : Atl) {x y : (bouquetPag X Y).E}
    (f : x --> y) :
    Set.range (bouquetRootIndex X Y x.1.unop x.2) subset
      Set.range (bouquetRootIndex X Y y.1.unop y.2) := by
  intro z hz
  rcases x with <|<|mx|>, vx|>
  rcases y with <|<|my|>, vy|>
  rcases hz with <|a, ha|>
  rw [<- ha]
  induction mx using Fin.cases with
  | zero =>
    induction my using Fin.cases with
    | zero => exact <|a, rfl|>
    | succ p =>
      have hf : p.succ -->
          (<|0, bouquetLength_pos X.P.folio Y.P.folio|> :
            Fin (bouquetLength X.P.folio Y.P.folio)) :=
        Quiver.Hom.unop f.val
      have h := leOfHom hf
      change p.1 + 1 <= 0 at h
      omega
  | succ n =>
    induction my using Fin.cases with
    | zero => exact <|bouquetRootIndex X Y n.succ vx a, rfl|>
    | succ p =>
      have hf : p.succ --> n.succ := Quiver.Hom.unop f.val
      have hpn : p <= n := Fin.succ_le_succ_iff.mp (leOfHom hf)
      let hxp : X.P.folio.paddedIndex p.1 <= X.P.folio.paddedIndex n.1 :=
        X.P.folio.paddedIndex_mono hpn
      let hyp : Y.P.folio.paddedIndex p.1 <= Y.P.folio.paddedIndex n.1 :=
        Y.P.folio.paddedIndex_mono hpn
      generalize hvx : ofLex vx = s
      rcases s with k | k
      . have evx : vx = toLex (Sum.inl k) :=
          ofLex.injective (by simpa using hvx)
        subst vx
        let k' := X.P.H.map (homOfLE hxp).op k
        have ef : f.val = (homOfLE (show p.succ <= n.succ from
            Fin.succ_le_succ_iff.mpr hpn)).op := Subsingleton.elim _ _
        have hfv : (bouquetPag X Y).H.map f.val (toLex (Sum.inl k)) = vy :=
          f.property
        have hvy : ofLex vy = Sum.inl k' := by
          rw [<- hfv, ef]
          change ofLex (toLex (Sum.map _ _ (ofLex (toLex (Sum.inl k))))) = _
          simp [k']
          congr 2
        have evy : vy = toLex (Sum.inl k') :=
          ofLex.injective (by simpa using hvy)
        subst vy
        let q := CategoryOfElements.homMk
```

```
          (X.P.cell (op (X.P.folio.paddedIndex n.1)) k)
          (X.P.cell (op (X.P.folio.paddedIndex p.1)) k')
          (homOfLE hxp).op rfl
        refine <|X.G.map q a, ?_|>
        change Sum.inl (X.G.map (X.P.cellToOrigin _ k') (X.G.map q a)) =
          Sum.inl (X.G.map (X.P.cellToOrigin _ k) a)
        apply congrArg Sum.inl
        have hc : q >> X.P.cellToOrigin _ k' = X.P.cellToOrigin _ k :=
          CategoryOfElements.ext X.P.H _ _ (Subsingleton.elim _ _)
        have hm := X.G.map_comp q (X.P.cellToOrigin _ k')
        rw [hc] at hm
        exact congrFun (congrArg Function.Embedding.toFun hm.symm) a
      . have evx : vx = toLex (Sum.inr k) :=
          ofLex.injective (by simpa using hvx)
        subst vx
        let k' := Y.P.H.map (homOfLE hyp).op k
        have ef : f.val = (homOfLE (show p.succ <= n.succ from
            Fin.succ_le_succ_iff.mpr hpn)).op := Subsingleton.elim _ _
        have hfv : (bouquetPag X Y).H.map f.val (toLex (Sum.inr k)) = vy :=
          f.property
        have hvy : ofLex vy = Sum.inr k' := by
          rw [<- hfv, ef]
          change ofLex (toLex (Sum.map _ _ (ofLex (toLex (Sum.inr k))))) = _
          simp [k']
          congr 2
        have evy : vy = toLex (Sum.inr k') :=
          ofLex.injective (by simpa using hvy)
        subst vy
        let q := CategoryOfElements.homMk
          (Y.P.cell (op (Y.P.folio.paddedIndex n.1)) k)
          (Y.P.cell (op (Y.P.folio.paddedIndex p.1)) k')
          (homOfLE hyp).op rfl
        refine <|Y.G.map q a, ?_|>
        change Sum.inr (Y.G.map (Y.P.cellToOrigin _ k') (Y.G.map q a)) =
          Sum.inr (Y.G.map (Y.P.cellToOrigin _ k) a)
        apply congrArg Sum.inr
        have hc : q >> Y.P.cellToOrigin _ k' = Y.P.cellToOrigin _ k :=
          CategoryOfElements.ext Y.P.H _ _ (Subsingleton.elim _ _)
        have hm := Y.G.map_comp q (Y.P.cellToOrigin _ k')
        rw [hc] at hm
        exact congrFun (congrArg Function.Embedding.toFun hm.symm) a

@[simp]
theorem imageDom.map_val {A B R : DomIns} (f : A --> R) (g : B --> R)
    (h : Set.range f subset Set.range g) (z : imageDom f) :
    (imageDom.map f g h z).1 = z.1 := rfl

def bouquetImageMap (X Y : Atl) {x y : (bouquetPag X Y).E} (f : x --> y) :
    bouquetDom X Y x --> bouquetDom X Y y :=
  imageDom.map _ _ (bouquetRange_mono X Y f)

def bouquetFunctor (X Y : Atl) : (bouquetPag X Y).E ==> DomIns where
  obj := bouquetDom X Y
  map := bouquetImageMap X Y
  map_id := by
    intro x
    apply DomIns.hom_ext
    intro z
    apply Subtype.ext
    change z.1 = z.1
    rfl
  map_comp := by
    intro x y z f g
    apply DomIns.hom_ext
    intro w
    apply Subtype.ext
    change w.1 = w.1
    rfl

def bouquetAtlas (X Y : Atl) : Atl where
  P := bouquetPag X Y
  G := bouquetFunctor X Y
  disjoint := by
```

```
    intro m i j hij x y hxy
    rcases m with <|m|>
    induction m using Fin.cases with
    | zero =>
      exact hij ((bouquetFolio X.P.folio Y.P.folio).originEquiv.injective
        (Subsingleton.elim _ _))
    | succ n =>
      rcases x.2 with <|a, ha|>
      rcases y.2 with <|b, hb|>
      have huv : x.1 = y.1 := by
        have e := congrArg (fun q => q.1) hxy
        change x.1 = y.1 at e
        exact e
      have hroot : bouquetRootIndex X Y n.succ i a =
          bouquetRootIndex X Y n.succ j b := ha.trans (huv.trans hb.symm)
      generalize hi : ofLex i = si
      generalize hj : ofLex j = sj
      rcases si with k | k <;> rcases sj with l | l
      . have ei : i = toLex (Sum.inl k) :=
          ofLex.injective (by simpa using hi)
        have ej : j = toLex (Sum.inl l) :=
          ofLex.injective (by simpa using hj)
        subst i
        subst j
        have hkl : k != l := by
          intro e
          subst l
          exact hij rfl
        change Sum.inl (X.G.map (X.P.cellToOrigin _ k) a) =
          Sum.inl (X.G.map (X.P.cellToOrigin _ l) b) at hroot
        exact X.disjoint _ k l hkl a b (Sum.inl.inj hroot)
      . have ei : i = toLex (Sum.inl k) :=
          ofLex.injective (by simpa using hi)
        have ej : j = toLex (Sum.inr l) :=
          ofLex.injective (by simpa using hj)
        subst i
        subst j
        change Sum.inl (X.G.map (X.P.cellToOrigin _ k) a) =
          Sum.inr (Y.G.map (Y.P.cellToOrigin _ l) b) at hroot
        exact Sum.noConfusion hroot
      . have ei : i = toLex (Sum.inr k) :=
          ofLex.injective (by simpa using hi)
        have ej : j = toLex (Sum.inl l) :=
          ofLex.injective (by simpa using hj)
        subst i
        subst j
        change Sum.inr (Y.G.map (Y.P.cellToOrigin _ k) a) =
          Sum.inl (X.G.map (X.P.cellToOrigin _ l) b) at hroot
        exact Sum.noConfusion hroot
      . have ei : i = toLex (Sum.inr k) :=
          ofLex.injective (by simpa using hi)
        have ej : j = toLex (Sum.inr l) :=
          ofLex.injective (by simpa using hj)
        subst i
        subst j
        have hkl : k != l := by
          intro e
          subst l
          exact hij rfl
        change Sum.inr (Y.G.map (Y.P.cellToOrigin _ k) a) =
          Sum.inr (Y.G.map (Y.P.cellToOrigin _ l) b) at hroot
        exact Y.disjoint _ k l hkl a b (Sum.inr.inj hroot)

/-- The object-level atlas merge used by the horizontal construction. -/
def AtlMerge (X Y : Atl) : Atl := bouquetAtlas X Y

def bouquetTallPred (X Y : Atl) (n : Nat) :
    Fin (bouquetDepth X.P.folio Y.P.folio) :=
  <|min n (bouquetDepth X.P.folio Y.P.folio - 1), by
    have hp : 0 < bouquetDepth X.P.folio Y.P.folio :=
      lt_of_lt_of_le X.P.folio.positive (le_max_left _ _)
    omega|>

```

```
theorem bouquet_padded_succ (X Y : Atl) (n : Nat) :
    (bouquetFolio X.P.folio Y.P.folio).paddedIndex (n + 1) =
      (bouquetTallPred X Y n).succ := by
  apply Fin.ext
  simp only [Folio.paddedIndex, bouquetFolio, bouquetLength, bouquetTallPred,
    bouquetDepth, Fin.val_succ]
  have hp : 0 < max X.P.folio.length Y.P.folio.length :=
    lt_of_lt_of_le X.P.folio.positive (le_max_left _ _)
  omega

theorem left_padded_tallPred (X Y : Atl) (n : Nat) :
    X.P.folio.paddedIndex (bouquetTallPred X Y n).1 =
      X.P.folio.paddedIndex n := by
  apply Fin.ext
  simp only [Folio.paddedIndex, bouquetTallPred, bouquetDepth]
  have hx : X.P.folio.length <= max X.P.folio.length Y.P.folio.length :=
    le_max_left _ _
  omega

theorem right_padded_tallPred (X Y : Atl) (n : Nat) :
    Y.P.folio.paddedIndex (bouquetTallPred X Y n).1 =
      Y.P.folio.paddedIndex n := by
  apply Fin.ext
  simp only [Folio.paddedIndex, bouquetTallPred, bouquetDepth]
  have hy : Y.P.folio.length <= max X.P.folio.length Y.P.folio.length :=
    le_max_right _ _
  omega

/-- The finite coherent implementation of `AtlMerge` presents exactly the
componentwise positive pages used by `tallMerge`. -/
def AtlMerge.pageEquiv (X Y : Atl) (n : Nat) :
    (AtlMerge X Y).page (n + 1) Equiv Sum (X.page n) (Y.page n) := by
  change (bouquetChain X.P.folio Y.P.folio
      ((bouquetFolio X.P.folio Y.P.folio).paddedIndex (n + 1))).Obj Equiv _
  rw [bouquet_padded_succ]
  change Lex (Sum
    ((X.P.folio.core.obj (X.P.folio.paddedIndex
      (bouquetTallPred X Y n).1)).unop.Obj)
    ((Y.P.folio.core.obj (Y.P.folio.paddedIndex
      (bouquetTallPred X Y n).1)).unop.Obj)) Equiv _
  rw [left_padded_tallPred, right_padded_tallPred]
  exact ofLex

/-- The left input as a page-preserving subobject of the tall presentation of
an atlas merge.  In particular, occurrence `n` is sent to occurrence `n+1`;
no finite representative is selected here. -/
def tallBouquetLeftElement (X Y : Atl) (x : X.tall.E) : (AtlMerge X Y).tall.E := by
  refine <|op (x.1.unop + 1), ?_|>
  change (bouquetChain X.P.folio Y.P.folio
    ((bouquetFolio X.P.folio Y.P.folio).paddedIndex (x.1.unop + 1))).Obj
  rw [bouquet_padded_succ]
  change Lex (Sum
    ((X.P.folio.core.obj (X.P.folio.paddedIndex
      (bouquetTallPred X Y x.1.unop).1)).unop.Obj)
    ((Y.P.folio.core.obj (Y.P.folio.paddedIndex
      (bouquetTallPred X Y x.1.unop).1)).unop.Obj))
  rw [left_padded_tallPred]
  exact toLex (Sum.inl x.2)

/-- The right input in the tall presentation of an atlas merge. -/
def tallBouquetRightElement (X Y : Atl) (y : Y.tall.E) : (AtlMerge X Y).tall.E := by
  refine <|op (y.1.unop + 1), ?_|>
  change (bouquetChain X.P.folio Y.P.folio
    ((bouquetFolio X.P.folio Y.P.folio).paddedIndex (y.1.unop + 1))).Obj
  rw [bouquet_padded_succ]
  change Lex (Sum
    ((X.P.folio.core.obj (X.P.folio.paddedIndex
      (bouquetTallPred X Y y.1.unop).1)).unop.Obj)
    ((Y.P.folio.core.obj (Y.P.folio.paddedIndex
      (bouquetTallPred X Y y.1.unop).1)).unop.Obj))
  rw [right_padded_tallPred]
  exact toLex (Sum.inr y.2)

```

```
/-- The category of Atlas Federations used for iterated stable atlas merging. -/
abbrev AtlFed := StableAtlasFamily

/-- The horizontal bifunctor is federation merge on objects and applies a
stable traversal independently to every tagged component on arrows. -/
def AtlHorSum : AtlFed * AtlFed ==> AtlFed := StableAtlHorSum

def AtlBrd (X Y : AtlFed) :
    AtlHorSum.obj (X, Y) Iso AtlHorSum.obj (Y, X) :=
  stableAtlasBraiding X Y

def AtlAsoc (X Y Z : AtlFed) :
    AtlHorSum.obj (AtlHorSum.obj (X, Y), Z) Iso
      AtlHorSum.obj (X, AtlHorSum.obj (Y, Z)) :=
  stableAtlasAssociator X Y Z

def AtlLu (X : AtlFed) : AtlHorSum.obj (stableAtlasUnit, X) Iso X :=
  stableAtlasLeftUnitor X

def AtlRu (X : AtlFed) : AtlHorSum.obj (X, stableAtlasUnit) Iso X :=
  stableAtlasRightUnitor X

theorem horizontalLemma : Nonempty (SymmetricCategory AtlFed) :=
  <|inferInstance|>

instance : SymmetricCategory StableAtlasFamilyop where
  symmetry X Y := by
    apply Quiver.Hom.unop_inj
    simp

theorem staDaTravInc_essImage (F : StaDaTravPresheaf) :
    StaDaTravInc.essImage F :=
  <|<|F, trivial|>, <|Iso.refl _|>|>

instance staDaTravPreservesTensorRightForTensor (v : Type 3)
    (d : StableAtlasFamilyop) :
    PreservesColimitsOfShape
      (CostructuredArrow (MonoidalCategory.tensor StableAtlasFamilyop) d)
      (MonoidalCategory.tensorRight v) :=
  preservesColimitsOfShape_of_natIso
    (BraidedCategory.tensorLeftIsoTensorRight v)

instance staDaTravPreservesTensorRightForUnit (v : Type 3)
    (d : StableAtlasFamilyop) :
    PreservesColimitsOfShape
      (CostructuredArrow
        (Functor.fromPUnit
          (MonoidalCategory.tensorUnit StableAtlasFamilyop)) d)
      (MonoidalCategory.tensorRight v) :=
  preservesColimitsOfShape_of_natIso
    (BraidedCategory.tensorLeftIsoTensorRight v)

instance staDaTravPreservesTensorRightForUnitProduct (v : Type 3)
    (d : StableAtlasFamilyop * StableAtlasFamilyop) :
    PreservesColimitsOfShape
      (CostructuredArrow
        ((Functor.id StableAtlasFamilyop).prod
          (Functor.fromPUnit
            (MonoidalCategory.tensorUnit StableAtlasFamilyop))) d)
      (MonoidalCategory.tensorRight v) :=
  preservesColimitsOfShape_of_natIso
    (BraidedCategory.tensorLeftIsoTensorRight v)

instance staDaTravPreservesTensorRightForTensorProduct (v : Type 3)
    (d : StableAtlasFamilyop * StableAtlasFamilyop) :
    PreservesColimitsOfShape
      (CostructuredArrow
        ((MonoidalCategory.tensor StableAtlasFamilyop).prod
          (Functor.id StableAtlasFamilyop)) d)
      (MonoidalCategory.tensorRight v) :=
  preservesColimitsOfShape_of_natIso
    (BraidedCategory.tensorLeftIsoTensorRight v)

```

```
noncomputable def staDaTravMonoidal : MonoidalCategory StaDaTrav :=
  MonoidalCategory.monoidalOfHasDayConvolutions StaDaTravInc
    (ObjectProperty.fullyFaithfuliota IsStableDataTransversal)
    (fun _ _ => staDaTravInc_essImage _)
    (staDaTravInc_essImage _)

noncomputable instance : MonoidalCategory StaDaTrav := staDaTravMonoidal

noncomputable instance staDaTravLawful :
    MonoidalCategory.LawfulDayConvolutionMonoidalCategoryStruct
      StableAtlasFamilyop (Type 3) StaDaTrav :=
  MonoidalCategory.lawfulDayConvolutionMonoidalCategoryStructOfHasDayConvolutions
    StaDaTravInc (ObjectProperty.fullyFaithfuliota IsStableDataTransversal)
    (fun _ _ => staDaTravInc_essImage _)
    (staDaTravInc_essImage _)

noncomputable instance staDaTravDayConvolution (F G : StaDaTrav) :
    MonoidalCategory.DayConvolution (StaDaTravInc.obj F) (StaDaTravInc.obj G) :=
  MonoidalCategory.LawfulDayConvolutionMonoidalCategoryStruct.convolution
    StableAtlasFamilyop (Type 3) StaDaTrav F G

noncomputable instance staDaTravDayConvolutionRightNested (F G H : StaDaTrav) :
    MonoidalCategory.DayConvolution (StaDaTravInc.obj F)
      (MonoidalCategory.DayConvolution.convolution
        (StaDaTravInc.obj G) (StaDaTravInc.obj H)) :=
  MonoidalCategory.LawfulDayConvolutionMonoidalCategoryStruct.convolution_2
    StableAtlasFamilyop (Type 3) StaDaTrav F G H

noncomputable instance staDaTravDayConvolutionLeftNested (F G H : StaDaTrav) :
    MonoidalCategory.DayConvolution
      (MonoidalCategory.DayConvolution.convolution
        (StaDaTravInc.obj F) (StaDaTravInc.obj G))
      (StaDaTravInc.obj H) :=
  MonoidalCategory.LawfulDayConvolutionMonoidalCategoryStruct.convolution_2'
    StableAtlasFamilyop (Type 3) StaDaTrav F G H

noncomputable def staDaTravBraiding (F G : StaDaTrav) :
    MonoidalCategory.tensorObj F G Iso MonoidalCategory.tensorObj G F := by
  exact (ObjectProperty.fullyFaithfuliota IsStableDataTransversal).preimageIso
    (MonoidalCategory.DayConvolution.braiding
      (StaDaTravInc.obj F) (StaDaTravInc.obj G))

theorem staDaTravInc_map_braiding_hom (F G : StaDaTrav) :
    StaDaTravInc.map (staDaTravBraiding F G).hom =
      (MonoidalCategory.DayConvolution.braiding
        (StaDaTravInc.obj F) (StaDaTravInc.obj G)).hom := by
  exact (ObjectProperty.fullyFaithfuliota IsStableDataTransversal).map_preimage _

theorem staDaTravInc_map_tensorHom {F_1 F_2 G_1 G_2 : StaDaTrav}
    (f : F_1 --> F_2) (g : G_1 --> G_2) :
    StaDaTravInc.map (MonoidalCategory.tensorHom f g) =
      MonoidalCategory.DayConvolution.map
        (StaDaTravInc.map f) (StaDaTravInc.map g) := by
  simpa [StaDaTravInc, staDaTravLawful] using
    (MonoidalCategory.LawfulDayConvolutionMonoidalCategoryStruct.iota_map_tensorHom_hom_eq_tensorHom
      StableAtlasFamilyop (Type 3) StaDaTrav f g)

theorem staDaTravInc_map_associator_hom (F G H : StaDaTrav) :
    StaDaTravInc.map (MonoidalCategory.associator F G H).hom =
      (MonoidalCategory.DayConvolution.associator
        (StaDaTravInc.obj F) (StaDaTravInc.obj G) (StaDaTravInc.obj H)).hom := by
  simpa [StaDaTravInc, staDaTravLawful] using
    (MonoidalCategory.LawfulDayConvolutionMonoidalCategoryStruct.
     iota_map_associator_hom_eq_associator_hom
      StableAtlasFamilyop (Type 3) StaDaTrav F G H)

noncomputable instance staDaTravBraided : BraidedCategory StaDaTrav where
  braiding := staDaTravBraiding
  braiding_naturality_right := fun X {_ _} f => by
    rw [<- MonoidalCategory.id_tensorHom, <- MonoidalCategory.tensorHom_id]
    apply (ObjectProperty.fullyFaithfuliota IsStableDataTransversal).map_injective
    simp only [Functor.map_comp, staDaTravInc_map_tensorHom,
      staDaTravInc_map_braiding_hom]
```

```
    exact MonoidalCategory.DayConvolution.braiding_naturality_right
      (StaDaTravInc.obj X) (StaDaTravInc.map f)
  braiding_naturality_left := fun {_ _} f Z => by
    rw [<- MonoidalCategory.tensorHom_id, <- MonoidalCategory.id_tensorHom]
    apply (ObjectProperty.fullyFaithfuliota IsStableDataTransversal).map_injective
    simp only [Functor.map_comp, staDaTravInc_map_tensorHom,
      staDaTravInc_map_braiding_hom]
    exact MonoidalCategory.DayConvolution.braiding_naturality_left
      (StaDaTravInc.map f) (StaDaTravInc.obj Z)
  hexagon_forward := fun X Y Z => by
    apply (ObjectProperty.fullyFaithfuliota IsStableDataTransversal).map_injective
    simp only [Functor.map_comp, staDaTravInc_map_associator_hom,
      staDaTravInc_map_braiding_hom]
    rw [<- MonoidalCategory.tensorHom_id, <- MonoidalCategory.id_tensorHom]
    simp only [staDaTravInc_map_tensorHom]
    exact MonoidalCategory.DayConvolution.hexagon_forward
      (StaDaTravInc.obj X) (StaDaTravInc.obj Y) (StaDaTravInc.obj Z)
  hexagon_reverse := fun X Y Z => by
    apply (ObjectProperty.fullyFaithfuliota IsStableDataTransversal).map_injective
    simp only [Functor.map_comp, staDaTravInc_map_braiding_hom]
    rw [<- MonoidalCategory.id_tensorHom, <- MonoidalCategory.tensorHom_id]
    simp only [staDaTravInc_map_tensorHom]
    exact MonoidalCategory.DayConvolution.hexagon_reverse
      (StaDaTravInc.obj X) (StaDaTravInc.obj Y) (StaDaTravInc.obj Z)

set_option maxHeartbeats 2400000 in
noncomputable instance : SymmetricCategory StaDaTrav where
  symmetry F G := by
    apply (ObjectProperty.fullyFaithfuliota IsStableDataTransversal).map_injective
    simp only [Functor.map_comp]
    exact MonoidalCategory.DayConvolution.symmetry
      (StaDaTravInc.obj F) (StaDaTravInc.obj G)

/-- `StaDaTravMon` is definitionally the category of stable data transversals
equipped with the Day convolution instance above. -/
abbrev StaDaTravMon := StaDaTrav

noncomputable def I : StaDaTravMon := MonoidalCategory.tensorUnit StaDaTravMon

noncomputable def I_dayConvolutionUnit :
    MonoidalCategory.DayConvolutionUnit (StaDaTravInc.obj I) :=
  MonoidalCategory.LawfulDayConvolutionMonoidalCategoryStruct.convolutionUnit
    StableAtlasFamilyop (Type 3) StaDaTravMon

noncomputable def HorSum : StaDaTravMon * StaDaTravMon ==> StaDaTravMon :=
  MonoidalCategory.tensor StaDaTravMon

noncomputable def Brd (F G : StaDaTravMon) :
    MonoidalCategory.tensorObj F G Iso MonoidalCategory.tensorObj G F :=
  staDaTravBraiding F G

noncomputable def Asoc (F G H : StaDaTravMon) :
    MonoidalCategory.tensorObj (MonoidalCategory.tensorObj F G) H Iso
      MonoidalCategory.tensorObj F (MonoidalCategory.tensorObj G H) :=
  MonoidalCategory.associator F G H

noncomputable def Lu (F : StaDaTravMon) :
    MonoidalCategory.tensorObj I F Iso F := MonoidalCategory.leftUnitor F

noncomputable def Ru (F : StaDaTravMon) :
    MonoidalCategory.tensorObj F I Iso F := MonoidalCategory.rightUnitor F

theorem serenityLemma : Nonempty (SymmetricCategory StaDaTravMon) :=
  <|inferInstance|>
end

end Datra
```

Listing 1: Complete Lean formalization of DaTra